

Introduction [\(Ask a Question\)](#)

CoreAXI4ProtoConv is designed for high throughput data transfer between the AXI4 memory-mapped and AXI4-stream interfaces. CoreAXI4ProtoConv supports the Memory Mapped to Stream (MM2S) and Stream to Memory Mapped (S2MM) conversion independently in a full-duplex like method. AXI4-Lite target interface is used for register access.

CoreAXI4ProtoConv Summary [\(Ask a Question\)](#)

Core Version	This document applies to CoreAXI4ProtoConv v3.0.
Supported Device Families	• PolarFire[®] SoC • PolarFire • RT PolarFire • RTG4[™] • IGLOO[®] 2 • SmartFusion[®] 2
Supported Tool Flow	Requires Libero [®] SoC v12.0 or later releases.
Supported Interfaces	• AXI4 Lite: Dedicated for S2MM block to access S2MM registers. Similarly, dedicated for MM2S block to access MM2S registers. • AXI4 Stream: Dedicated for S2MM block to receive stream data from external AXI4-Stream initiator interface. Similarly, AXI4 Stream initiator interfaces dedicated for MM2S block to transmit data to external AXI4-Stream target interface. • AXI4: Dedicated for S2MM block to transmit data to external AXI4 target interface. Similarly, dedicated for MM2S block to receive data from external AXI4 target interface. For more information, see Functional Description.
Licensing	CoreAXI4ProtoConv is license free. Un-obfuscated Verilog RTL source files provided.
Installation Instructions	CoreAXI4ProtoConv must be installed to the Libero SoC IP Catalog automatically through the update function. Alternatively, CoreAXI4ProtoConv could be manually downloaded from the catalog. Once the IP core is installed, it is configured, generated, and instantiated within SmartDesign for inclusion in the project.
Device Utilization and Performance	A summary of utilization and performance information for CoreAXI4ProtoConv is listed in the Device Utilization and Performance .

CoreAXI4ProtoConv Change Log Information

This section provides a comprehensive overview of the newly incorporated features, beginning with the most recent release. For more information about the problems resolved, see the [Resolved Issues](#) section.

Version	What's New
v3.0	- Added support for Interrupt. - Added support for S2MM packet drop due to overflow or error. - Added support for statistics counter when errors or packet is dropped. - Added support for unaligned length for the Length register when MM2S is enabled.
v2.0	First production release of CoreAXI4ProtoConv

Table of Contents

Introduction.....	1
CoreAXI4ProtoConv Summary.....	1
1. Features.....	5
2. Overview.....	6
2.1. Stream to Memory Map (S2MM).....	6
2.2. Memory Map to Stream (MM2S).....	6
3. Functional Description.....	8
3.1. S2MM.....	8
3.2. MM2S.....	10
3.3. Latency.....	11
3.4. Programming Sequence.....	13
4. IP Core Parameters and Interface Signals.....	16
4.1. CoreAXI4ProtoConv Configurator Settings.....	16
4.2. Inputs and Outputs Signals.....	21
5. Timing Diagrams.....	28
5.1. S2MM Timing Diagrams.....	28
5.2. MM2S Timing Diagrams.....	30
6. Implementation of CoreAXI4ProtoConv.....	33
6.1. SmartDesign.....	33
6.2. Verification.....	36
7. S2MM Register Map and Descriptions.....	40
7.1. S2MM_VERSN.....	41
7.2. S2MM_STS.....	42
7.3. S2MM_PKTDRPOVF_CNTR.....	44
7.4. S2MM_PKTDRPERR_CNTR.....	45
7.5. S2MM_LEN.....	46
7.6. S2MM_INTRSRC.....	47
7.7. S2MM_INTRENB.....	49
7.8. S2MM_CTRL.....	50
7.9. S2MM_AXI4BUSERR_CNTR.....	51
7.10. S2MM_ADDR1.....	52
7.11. S2MM_ADDR0.....	53
8. MM2S Register Map and Descriptions.....	54
8.1. MM2S_VERSN.....	55
8.2. MM2S_STS.....	56
8.3. MM2S_LEN.....	57
8.4. MM2S_INTRSRC.....	58
8.5. MM2S_INTRENB.....	59
8.6. MM2S_CTRL.....	60
8.7. MM2S_ADDR1.....	61

8.8.	MM2S_AXI4BUSERR_CNTR.....	62
8.9.	MM2S_ADDR0.....	63
9.	Additional References.....	64
9.1.	Known Issues and Workarounds.....	64
9.2.	Discontinued Features and Devices.....	64
10.	Device Utilization and Performance.....	65
10.1.	CoreAXI4ProtoConv S2MM Device Utilization and Performance.....	65
10.2.	CoreAXI4ProtoConv MM2S Device and Utilization Performance.....	79
11.	Glossary.....	85
12.	Resolved Issues.....	86
13.	Revision History.....	87
	Microchip FPGA Support.....	88
	Microchip Information.....	88
	Trademarks.....	88
	Legal Notice.....	88
	Microchip Devices Code Protection Feature.....	89

1. Features [\(Ask a Question\)](#)

CoreAXI4ProtoConv supports the following key features:

- The CoreAXI4ProtoConv core is fully Compliant with the AXI4-Lite Interface, AXI4 Interface, and the AXI4-Stream Interface.
- Configurable AXI4 and AXI4-Stream Data Width
 - AXI4/AXI4-Stream: 32, 64, 128, 256, and 512
- Configurable Address Width
- Configurable In-determinate AXI4-Stream Transfers when S2MM is Enabled
- Configurable Little Endian to Big Endian Conversion
- Configurable FIFO Buffers and Configurable Packet-based FIFO (Store and Forward)
- Configurable Buffer Depth
- Interrupt Mechanism for Processor to Control the Data Transfers Efficiently
- Packet Drop due to Error and Packet Drop due to Overflow.
- Configurable Reset Type and RAM Type

2. Overview [\(Ask a Question\)](#)

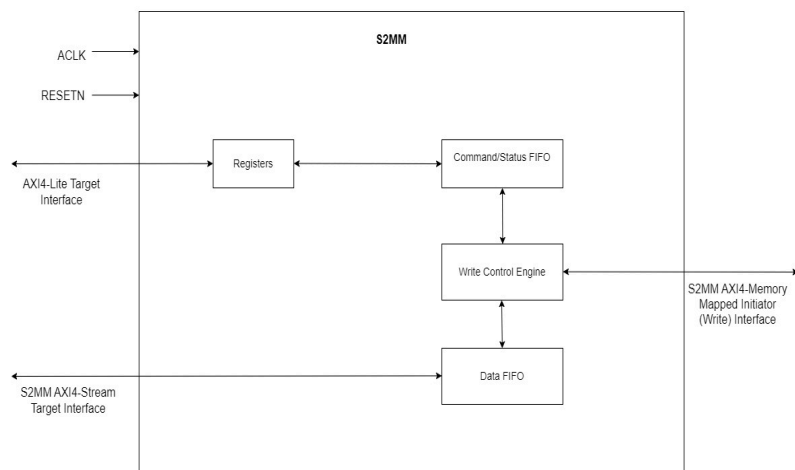
The CoreAXI4ProtoConv core is key building block to convert AXI4-Stream protocol to AXI4 and vice versa with 4 KB address boundary projection and automatic burst partitioning.

CoreAXI4ProtoConv has two main blocks, that is, S2MM and MM2S.

2.1 Stream to Memory Map (S2MM) [\(Ask a Question\)](#)

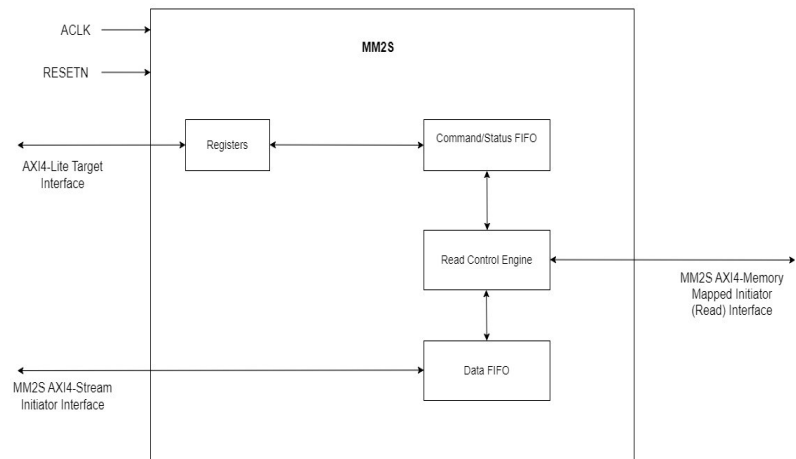
The S2MM block is responsible for transmitting transactions between the AXI4-Stream Interface and AXI4-Memory Mapped. It has its dedicated sets of registers (descriptor) to load command and read the status of corresponding commands. S2MM block supports queuing both commands and status. Registers can be accessed through AXI4-Lite interface. Based upon the command inputs and input data from the AXI4-Stream interface, the S2MM block issues a write request on the AXI4-Memory Mapped interface. Input stream data can be optionally stored inside a data FIFO. Data FIFO supports optional store and forward mode or packet mode in which data is not read from the FIFO until complete packet is received from the AXI4-Stream interface that is, TLAST of AXI4-Stream interface is asserted.

Figure 2-1. S2MM Block Diagram



2.2 Memory Map to Stream (MM2S) [\(Ask a Question\)](#)

The MM2S block is responsible for transmitting transactions between the AXI4-Memory Mapped Interface and AXI4-Stream Interface. It has its dedicated sets of registers (descriptor) to load command and read the status of corresponding commands. MM2S block supports queuing both commands and status. Registers can be accessed through AXI4-Lite interface. Based on command inputs, the MM2S block issues a read request on the AXI4-Memory Mapped interface. Read data can be optionally stored inside the data FIFO. Data FIFO supports optional store and forward mode or packet mode in which data is not read from the FIFO until complete packet is received from the AXI4-Memory Mapped interface that is, number of bytes configured in [MM2S Length Register](#) are received.

Figure 2-2. MM2S Block Diagram

3. Functional Description [\(Ask a Question\)](#)

This section describes the functional description of the CoreAXI4ProtoConv.

3.1 S2MM [\(Ask a Question\)](#)

This section describes the functional description of the S2MM block.

3.1.1 Command/Status FIFO [\(Ask a Question\)](#)

Launch the CoreAXI4ProtoConv Configurator using the Libero SoC Design Suite. Click the **S2MM Configuration** tab and select the **S2MM Command/Status FIFO Enable** option to instantiate the Command/Status FIFO for S2MM. Command/Status FIFO is required to push more than one command and its status into the command and status FIFO respectively.

Command/Status FIFO module supports the following features:

- Configurable FIFO depth
- Configurable memory type

3.1.2 Data FIFO [\(Ask a Question\)](#)

Launch the CoreAXI4ProtoConv Configurator using the Libero SoC Design Suite. Click the **S2MM Configuration** tab and select the **S2MM Data FIFO Enable** option to instantiate the Data FIFO for S2MM. Data FIFO is required to store the data coming from the AXI4-Stream interface.

Data FIFO module supports the following features:

- Configurable FIFO depth
- Configurable memory type
- Configurable FIFO mode - Cut Through mode or Store and Forward mode

When **S2MM Store and Forward FIFO Enable** (in **S2MM Configuration** tab) is not selected, FIFO operates in normal mode and reads data from the FIFO as soon as possible.

When **S2MM Store and Forward FIFO Enable** (in **S2MM Configuration** tab) is selected, FIFO operates in store and forward mode or packet mode in which data from the FIFO is not read until a complete packet is received from the AXI4-Stream that is, TLAST of AXI4-Stream interface is asserted. When **S2MM Store and Forward FIFO Enable** (in **S2MM Configuration** tab) is selected, you must configure **S2MM Data FIFO Depth** (in **S2MM Configuration** tab) to more than a packet size to avoid deadlock.

3.1.3 Registers [\(Ask a Question\)](#)

This block implements AXI4-Lite compliant target interface to access all the registers. For more details about S2MM registers, see [S2MM Register Map and Descriptions](#).

3.1.4 Write Control Engine [\(Ask a Question\)](#)

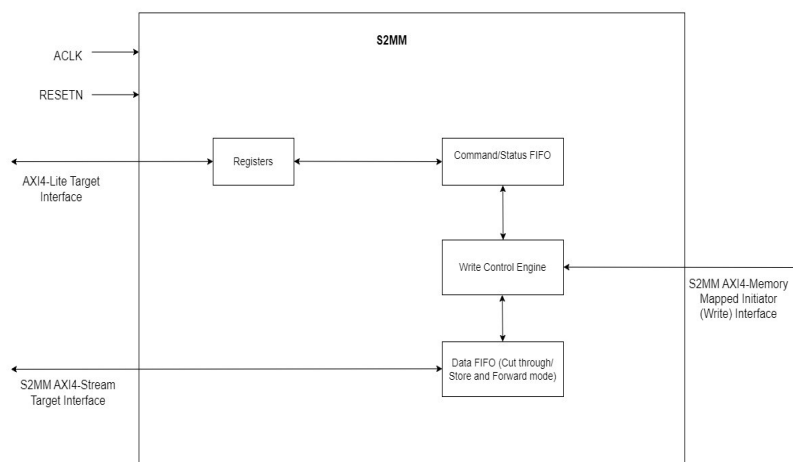
This block implements the AXI4-Memory Mapped initiator write channel interface. This block is responsible to break the AXI4-Memory Mapped initiator transaction into multiple transactions when the 4K address boundary is crossed or number of the bytes configured in the S2MM Length register is more than the supported burst length of the AXI4-Memory Mapped initiator interface burst type. For example, if S2MM Address0 register is configured to 8184, S2MM Length register is configured to 256 bytes and **S2MM_DATA_WIDTH** is set to 32 then Core breaks the one 256 bytes transaction into two transactions. For the first transaction, AXI4-Memory Mapped interface write address is set to 8184 and AXI4-Memory Mapped Initiator interface write burst length is set to 1 (2 beat transfer- 8 bytes). For the second transaction, AXI4-Memory Mapped interface write address is set to 8192 and AXI4-Memory Mapped Initiator interface write burst length is set to 61 (62 beat transfer- 248 bytes).

- When **S2MM Command/Status FIFO Enable** (in **S2MM Configuration** tab) is selected, the descriptors will be loaded into command FIFO and command FIFO is read when it is not

empty, and AXI4-Memory Mapped initiator write address channel information start address, burst length, and burst type are generated. Command FIFO is read again after the transaction is completed.

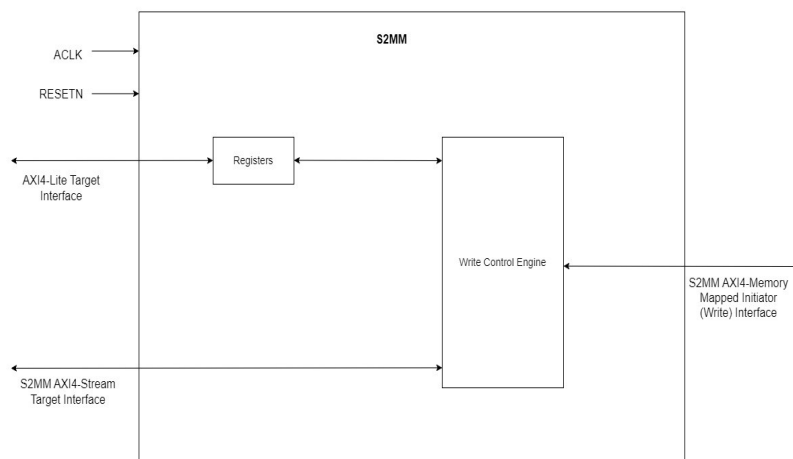
- When **S2MM Data FIFO Enable** (in **S2MM Configuration** tab) is selected, data received from the AXI4-Stream interface is stored into the data FIFO and data FIFO is read when it is not empty, and the AXI4-Memory Mapped initiator write data channel information is generated.
- When **S2MM Command/Status FIFO Enable** (in **S2MM Configuration** tab) is selected, status is stored in status FIFO on receiving the response on the AXI4-Memory Mapped initiator write response channel. When **S2MM Undefined Burst Length Enable** (in **S2MM Configuration** tab) is selected, along with the status, number of bytes received from the AXI4-Stream interface is also stored into the status FIFO.

Figure 3-1. When S2MM Command/Status FIFO Enable and S2MM Data FIFO Enable (Cut through/Store and Forward mode)



- To reduce resources, you can clear **S2MM Command/Status FIFO Enable** (in **S2MM Configuration** tab) and **S2MM Data FIFO Enable** (in **S2MM Configuration** tab) and bypass Command/Status and Data FIFO. When **S2MM Command/Status FIFO Enable** (in **S2MM Configuration** tab) is cleared, you must not write next command until the current command is executed.

Figure 3-2. When S2MM Command/Status FIFO Disable and S2MM Data FIFO Disable



3.2 MM2S [\(Ask a Question\)](#)

This section describes the functional description of the MM2S blocks.

3.2.1 Command/Status FIFO [\(Ask a Question\)](#)

Launch the CoreAXI4ProtoConv Configurator using the Libero SoC Design Suite. Click the **MM2S Configuration** tab and select the **MM2S Command/Status FIFO Enable** option to instantiate the Command/Status FIFO for MM2S. Command/Status FIFO is required to push more than one command and its status into the command and status FIFO respectively

Command/Status FIFO module supports the following features:

- Configurable FIFO depths
- Configurable memory type

3.2.2 Data FIFO [\(Ask a Question\)](#)

Launch the CoreAXI4ProtoConv configurator using the Libero[®] SoC Design Suite. To instantiate the Data FIFO for MM2S, click **MM2S Configuration** tab and select the **MM2S Data FIFO Enable** option. Data FIFO is required to store the data coming from the AXI4-Memory Mapped initiator read channel interface.

Data FIFO module supports the following features:

- Configurable FIFO depths
- Configurable memory type
- Configurable FIFO mode - Cut Through mode or Store and Forward mode

When **MM2S Store and Forward FIFO Enable** (in **MM2S Configuration** tab) is not selected, FIFO operates in normal mode and reads data from the FIFO as soon as possible.

When **MM2S Store and Forward FIFO Enable** (in **MM2S Configuration** tab) is selected, FIFO operates in store and forward mode or packet mode in which data from the FIFO will not be read until complete packet is received from the AXI4-Memory Mapped read channel interface that is, number of bytes configured in MM2S Length register are received. When **MM2S Store and Forward FIFO Enable** (in **MM2S Configuration** tab) is selected, you must configure **MM2S Data FIFO Depth** (in **MM2S Configuration** tab) more than packet size to avoid deadlock.

3.2.3 MM2S Registers [\(Ask a Question\)](#)

This block implements AXI4-Lite compliant target interface to access all the registers. For more details about MM2S registers, see [MM2S Register Map and Descriptions](#).

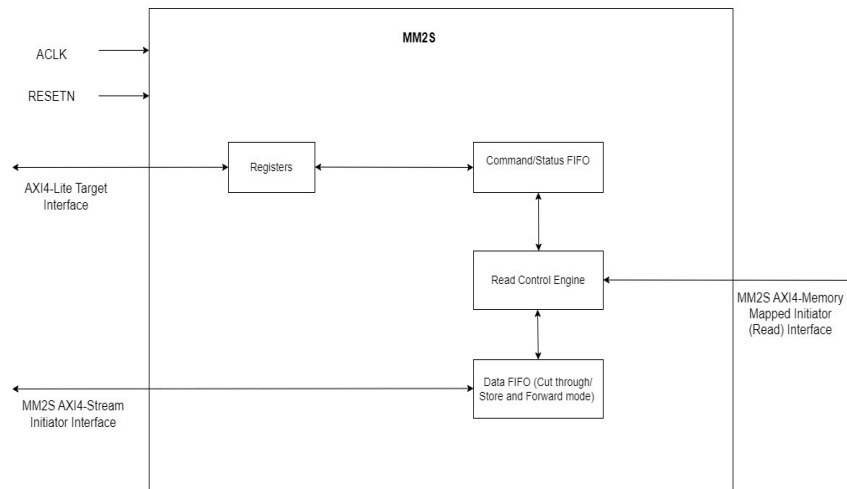
3.2.4 Read Control Engine [\(Ask a Question\)](#)

This block controls the AXI4-Memory Mapped initiator read channel interface. This block is responsible to break the AXI4-Memory Mapped initiator transaction into multiple transactions when the 4K address boundary is crossed or number of the bytes configured in the MM2S Length register is more than the supported burst length of the AXI4-Memory Mapped initiator interface burst type. For example, if MM2S Address0 register is configured to 8184, MM2S Length register is configured to 256 bytes and **MM2S_DATA_WIDTH** is set to 32 then Core breaks the one 256 bytes transaction into two transactions. For the first transaction, AXI4-Memory Mapped interface read address is set to 8184 and AXI4-Memory Mapped Initiator interface read burst length is set to 1 (2 beat transfer- 8 bytes). For the second transaction, AXI4-Memory Mapped interface read address is set to 8192 and AXI4-Memory Mapped Initiator interface read burst length is set to 61 (62 beat transfer- 248 bytes).

- When **MM2S Command/Status FIFO Enable** (in **MM2S Configuration** tab) is selected, descriptor is loaded into command FIFO and command FIFO is read when it is not empty and AXI4-Memory Mapped initiator read address channel information start address, burst length, and burst type are generated. The command FIFO is read again after the transaction is completed.

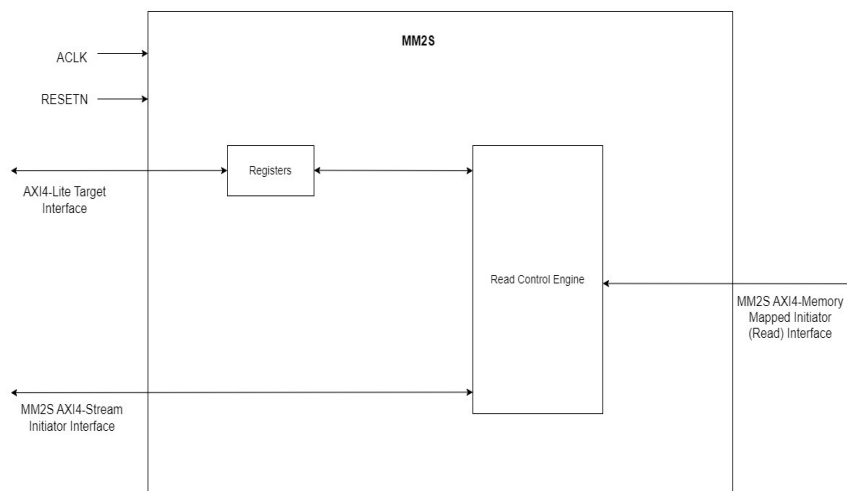
- When **MM2S Data FIFO Enable** (in **MM2S Configuration** tab) is selected, the AXI4-Memory Mapped initiator read data channel information is written into the data FIFO when it is not full.
- When **MM2S Command/Status FIFO Enable** (in **MM2S Configuration** tab) is selected, the status is stored in status FIFO on receiving the response on the AXI4-Memory Mapped initiator read response channel and data read from the FIFO is given to the AXI4-Stream initiator interface.

Figure 3-3. When MM2S Command/Status FIFO Enable and MM2S Data FIFO Enable (Cut through/Store and Forward mode)



- To reduce resources, you can clear **MM2S Command/Status FIFO Enable** (in **MM2S Configuration** tab) and **MM2S Data FIFO Enable** (in **MM2S Configuration** tab). When **MM2S Command/Status FIFO Enable** (in **MM2S Configuration** tab) is cleared, you must not write next command until the current command is executed.

Figure 3-4. When MM2S Command/Status FIFO Disable and MM2S Data FIFO Disable



3.3 Latency [\(Ask a Question\)](#)

Latency for S2MM and MM2S functional blocks are defined in the following tables.

Table 3-1. S2MM Latency

Interface	Latency (Clocks)	Description
AXI4 Lite target write	Minimum two	By default, T_AXI4L_AWREADY is asserted. T_AXI4L_WREADY is asserted after one clock cycle. T_AXI4L_AWVALID and T_AXI4L_WREADY are asserted. T_AXI4L_BVALID is asserted after T_AXI4L_WVALID and T_AXI4L_WREADY are asserted.
AXI4 Lite target read	Minimum two – When S2MM Status Register is read Minimum one – When other than S2MM Status Register is read	By default, T_AXI4L_ARREADY is asserted. T_AXI4L_RVALID is asserted after one clock cycle. T_AXI4L_ARVALID and T_AXI4L_ARREADY are asserted.
AXI4 Stream target	Minimum zero (0)	When S2MM Data FIFO Enable is selected, T_AXI4S_TREADY remains asserted until FIFO gets full. When S2MM Data FIFO Enable (in S2MM Configuration tab) is cleared, T_AXI4S_TREADY is controlled by I_S2MMAXI4_WREADY after the AXI4 Memory Mapped write address phase is completed.
AXI4 Memory Mapped initiator	Minimum eight when S2MM Command/Status FIFO Enable (in S2MM Configuration tab) is selected Minimum six when S2MM Command/Status FIFO Enable (in S2MM Configuration tab) is not selected	When S2MM Command/Status FIFO Enable (in S2MM Configuration tab) is selected, FIFO empty is de-asserted after one clock cycle of Start bit of S2MM Control Register is programmed. When S2MM Command/Status FIFO Enable is selected, I_S2MMAXI4_AWVALID is asserted after six clock cycle of FIFO empty is de-asserted. When S2MM Command/Status FIFO Enable (in S2MM Configuration tab) is cleared, I_S2MMAXI4_AWVALID is asserted after five clock cycle of Start bit of S2MM Control Register is programmed. I_S2MMAXI4_WVALID is asserted after one clock cycle of the completion of the AXI4 Memory Mapped write address phase. I_S2MMAXI4_BREADY is tied high.

Table 3-2. MM2S Latency

Interface	Latency (Clocks)	Description
AXI4 Lite target write	Minimum two	By default, T_AXI4L_AWREADY is asserted. T_AXI4L_WREADY is asserted after one clock cycle. T_AXI4L_AWVALID and T_AXI4L_WREADY are asserted. T_AXI4L_BVALID is asserted after T_AXI4L_WVALID and T_AXI4L_WREADY are asserted.
AXI4 Lite target read	Minimum two – When MM2S Status Register is read Minimum one – When other than MM2S Status Register is read	By default, T_AXI4L_ARREADY is asserted. T_AXI4L_RVALID is asserted after one clock cycle. T_AXI4L_ARVALID and T_AXI4L_ARREADY are asserted.
AXI4 Stream target	Minimum zero (0)	When MM2S Data FIFO Enable (in MM2S Configuration tab) is selected, I_AXI4S_TVALID remains asserted until FIFO gets empty. When MM2S Data FIFO Enable (in MM2S Configuration tab) is cleared, I_AXI4S_TVALID is controlled by I_MM2SAXI4_RVALID after the AXI4 Memory Mapped read address phase is completed.
AXI4 Memory Mapped initiator	Minimum eight when MM2S Command/Status FIFO Enable (in MM2S Configuration tab) is selected Minimum six when MM2S Command Status FIFO Enable (in MM2S Configuration tab) is selected	When MM2S Command/Status FIFO Enable (in MM2S Configuration tab) is selected, FIFO empty is de-asserted after one clock cycle of Start bit of MM2S Control Register is programmed. When MM2S Command/Status FIFO Enable (in MM2S Configuration tab) is selected, I_MM2SAXI4_ARVALID is asserted after six clock cycle of FIFO empty is de-asserted. When MM2S Command/Status FIFO Enable (in MM2S Configuration tab) is cleared, I_MM2SAXI4_ARVALID is asserted after five clock cycle of Start bit of MM2S Control Register is programmed. I_MM2SAXI4_RREADY is asserted after one clock cycle of the completion of the AXI4 Memory Mapped read address phase.

3.4 Programming Sequence [\(Ask a Question\)](#)

The following sections show the programming sequence for S2MM register, for various configuration settings. It is for reference only. The following programming sequence can also be used for MM2S by configuring the corresponding MM2S registers.

3.4.1 When S2MM Command FIFO is disabled and Interrupt is not used [\(Ask a Question\)](#)

To configure and manage S2MM registers, follow these steps:

1. Program the S2MM address 0 register.
2. Program the S2MM address 1 register (if the GUI parameter **S2MM Address Width** is set to more than 32).
3. Program the S2MM length register.
4. Program the S2MM control register.



Important: Start bit must be set to 1. Configure the other bits as per the requirement.

5. Wait until the Done bit of S2MM Status register is not set to 1.
6. Read the other status bits of the S2MM status register to determine the error status, or other information provided in status register.
7. Read the data from the external memory at the location programmed in steps 1 and 2.
8. Repeat steps 1-7.

3.4.2 When S2MM Command FIFO is enabled and Interrupt is not used [\(Ask a Question\)](#)

To configure and manage S2MM registers, follow these steps:

1. Program the S2MM address 0 register.
2. Program the S2MM address 1 register (if the GUI parameter **S2MM Address Width** is set to more than 32).
3. Program the S2MM length register.
4. Program the S2MM control register.



Important: Start bit must be set to 1. Configure the other bits as per the requirement.

5. Repeat steps 1-4 to program multiple descriptors as per the requirement.
6. Wait until the Done bit of the S2MM Status register is not set to 1.
7. Read the other status bits of the S2MM status register to determine the error status, or other information provided in the status register.
8. Increment the internal counter to count number of packets read.
9. Read the data from the external memory at the location programmed in steps 1 and 2.



Important: When multiple descriptors are configured then user must push the values configured in steps 1-3 into an internal queue and pop the queue when reading the data from the external memory.

10. Repeat steps 6-9 until the internal counter reaches the number of descriptors programmed in steps 1-4.
11. Repeat steps 1-10.

3.4.3 When S2MM Command FIFO is enabled and Interrupt is used [\(Ask a Question\)](#)

To configure and manage S2MM registers, follow these steps:

1. Program the S2MM interrupt enable register bit 0 to enable interrupt generation upon transfer completion. Configure the other bits of the interrupt enable register as per the requirement.
2. Program the S2MM address 0 register.
3. Program the S2MM address 1 register (if the GUI parameter **S2MM Address Width** is set to more than 32).
4. Program the S2MM length register.
5. Program the S2MM control register.



Important: Start bit must be set to 1. Configure the other bits as per the requirement.

6. Repeat steps 2-5. If S2MM_INT is asserted, execute the interrupt service routine. Within the interrupt service routine, perform the following actions:
 - a. Read the S2MM interrupt source register and store its value in the internal register to identify the cause of the interrupt.
 - b. Write one into the corresponding bit of S2MM interrupt source register to clear the interrupt.
 - c. If 0th bit of the internal register is '1', and all other error flags of the interrupt source register are zero, read data from the external memory at the location programmed in steps 1 and 2.



Important: When multiple descriptors are configured then user must push the values configured in steps 2-4 into the internal queue and pop the queue when reading data from the external memory.

- d. If any error flags are not zero, execute the error recovery program.

3.4.4 When S2MM Command FIFO is enabled, Undefined Burst Length is enabled and Interrupt is used [\(Ask a Question\)](#)

To configure and manage S2MM registers, follow these steps:

1. Program S2MM interrupt enable register bit 0 to enable interrupt generation upon packet transfer completion. Configure the other bits of the interrupt enable register as per the requirement.
2. Program the S2MM address 0 register.
3. Program the S2MM address 1 register (if the GUI parameter **S2MM Address Width** is set to more than 32).
4. Program the S2MM control register.



Important: Start bit must be set to 1. Configure the other bits as per the requirement.

5. Repeat steps 2-4. If S2MM_INT is asserted, execute the interrupt service routine. Within the interrupt service routine, perform the following actions:
 - a. Read the S2MM length register to identify the number of bytes received in the packet, and store this value in an internal variable.
 - b. Read the S2MM interrupt source register and store its value in an internal variable to identify the cause of the interrupt.
 - c. If 0th bit is set to 1, and other error flags in the interrupt source register are zero, read data from the external memory at the location programmed in steps 2 and 3.



Important: When multiple descriptors are configured then user must push the values configured in steps 2 and 3 into an internal queue and pop the queue when reading the data from the external memory.

- d. If error flags are not zero, then execute the error recovery program.

4. IP Core Parameters and Interface Signals [\(Ask a Question\)](#)

This section discusses the parameters in the **CoreAXI4ProtoConv** Configurator settings and I/O signals.

4.1 CoreAXI4ProtoConv Configurator Settings [\(Ask a Question\)](#)

The following table lists the CoreAXI4ProtoConv for configuring the core.



Important: The Name column in the following table shows the actual parameter names used in RTL. The Description column starts with the parameter names as they appear in the CoreAXI4ProtoConv Configurator.

Table 4-1. Configuration Parameters of CoreAXI4ProtoConv

Name	Valid Values	Default Values	Description
MM2S Parameters			
MM2S_ENABLE	0-1	1	MM2S Enable This parameter is used to enable/disable MM2S block as listed: 0: Disable MM2S block 1: Enable MM2S block
All the MM2S parameters described are configured only if MM2S Enable is enabled.			
MM2S_ADDR_WIDTH	32 64	32	MM2S Address Width This parameter is used to select address width for the address ports of AXI4 initiator and AXI4-stream initiator interface.
MM2S_DATA_WIDTH	32 64 128 256 512	32	MM2S Data Width (in bits) This parameter is used to select data width for the data ports of AXI4-Stream initiator and AXI4 initiator interface.
MM2S_DATA_FIFO_ENABLE	0-1	1	MM2S Data FIFO Enable This parameter is used to enable/disable data FIFO as listed: 0: Disable the data FIFO 1: Enable the data FIFO
MM2S_DATA_FIFO_DEPTH	8-32K	64	MM2S Data FIFO Depth This parameter is used to select data FIFO depth. This parameter can be configured only when MM2S Data FIFO Enable is enabled. Note: IP core internally derives the width of MM2S Data FIFO.
MM2S_DATA_RAM_TYPE	LSRAM μSRAM FABRIC	LSRAM	MM2S Data FIFO RAM Type This parameter is used to configure RAM implementation for the data FIFO. This parameter can be configured only if MM2S Data FIFO Enable is enabled.

.....continued

Name	Valid Values	Default Values	Description
MM2S_PKT_FIFO_ENABLE	Cut Through Store and Forward	Cut Through	MM2S Store and Forward FIFO Enable This parameter is used to select Cut Through mode or Store and Forward mode of data FIFO. When Store and forward mode is selected, data FIFO is not read until the last beat of AXI4 transfer is received. When Cut through mode is selected, data FIFO is read immediately that is, when data FIFO is not empty. When Store and Forward is selected, MM2S length register and MM2S Data FIFO Depth must be configured such that, data FIFO must be able to store number of bytes configured in MM2S length register. For example, if MM2S Data Width is configured to 64 (8 bytes) and MM2S Data FIFO Depth is configured to 2K, which can store 16K bytes. Then MM2S length register must not be configured to more than 16K bytes. This parameter can be configured only if MM2S Data FIFO Enable is enabled.
MM2S_ENDIAN_CONV	0-1	0	MM2S Enable Big to Little Endian Conversion 0: Disable endian conversion. This option must be selected when data input on the AXI4-Memory Mapped interface is in little-endian format. 1: Enable endian conversion. This option must be selected when data input on the AXI4-Memory Mapped interface is in big-endian format. Endian conversion is performed at byte level that is, Most Significant Byte (MSB) of the AXI4-Memory Mapped interface data is placed on Least Significant Byte (LSB) of AXI4-Stream interface data and similarly other bytes positions are changed. Endian conversion is performed for AXI4-Memory Mapped initiator read data and read user data ports (when MM2S User Enable is selected).
MM2S_CMDSTS_FIFO_ENABLE	0-1	1	MM2S Command/Status FIFO Enable This parameter is used to enable/disable command/status FIFO. When this parameter is enabled, the command FIFO, and the status FIFO are inferred by the IP Core. Command FIFO stores MM2S transfer request and status FIFO stores MM2S transfer read response. You can push MM2S transfer request into the command FIFO by writing descriptor. One descriptor consists MM2S Length register, MM2S Address0 register, MM2S Address1 register (if MM2S Address Width is set to more than 32), and MM2S control register. When start (0th bit) of the MM2S Control register is written, descriptor is stored into command FIFO. Similarly, for each descriptor, the read response received from the AXI4-Memory Mapped initiator is stored in status FIFO. IP core also stores the Done (0th bit) flag when the number of bytes configured in the MM2S length registers are received from the AXI4-Memory Mapped. You can read the status FIFO by reading MM2S status register.

.....continued

Name	Valid Values	Default Values	Description
MM2S_CMDSTS_FIFO_DEPTH	8 16 32 64 128 256 512 1024	16	Number of MM2S transfer request and response can be stored in Command/Status FIFO This parameter is used to select number of MM2S transfer request and response can be stored in command/status FIFO. This parameter can be configured only when MM2S Command/Status FIFO Enable is selected.
MM2S_CMDSTS_RAM_TYPE	LSRAM μSRAM Fabric	uSRAM	MM2S Command/Status FIFO RAM Type This parameter is used to configure RAM implementation for the command/status FIFO. This parameter can be configured only when MM2S Command/Status FIFO Enable is selected.
MM2S_USER_ENABLE	0-1	0	MM2S User Enable This parameter is used to enable or disable the AXI4-Stream and AXI4 interface user ports.
MM2S_USER_WIDTH	1-2048	1	MM2S User Width (in bits) This parameter is used to select width for user ports of AXI4-Stream initiator and AXI4-Memory Mapped initiator (read data channel). This parameter can be configured only when MM2S User Enable is selected. This parameter must be configured in multiple of eight when MM2SEnable Big to Little Endian Conversion is selected.
S2MM Parameters			
S2MM_ENABLE	0-1	1	S2MM Enable This parameter is used to enable or disable S2MM block as listed: 0: Disable S2MM block 1: Enable S2MM block
All the following S2MM parameters described can be configured only if S2MM Enable is enabled			
S2MM_ADDR_WIDTH	32 64	32	S2MM Address Width This parameter is used to select address width for the address ports of the AXI4-Memory Mapped initiator and the AXI4-Stream target interface.
S2MM_DATA_WIDTH	32 64 128 256 512	32	S2MM Data Width (in bits) This parameter is used to select data width for the data ports of AXI4-Stream target and AXI4-Memroy Mapped initiator interface.
S2MM_DATA_FIFO_ENABLE	0-1	1	S2MM Data FIFO Enable This parameter is used to enable or disable data FIFO as listed: 0: Disable the data FIFO 1: Enable the data FIFO
S2MM_DATA_FIFO_DEPTH	8-32K	64	S2MM Data FIFO Depth This parameter is used to select data FIFO depth. This parameter can be configured only when S2MM Data FIFO Enable is selected. Note: IP core internally derives the width of S2MM Data FIFO.

.....continued

Name	Valid Values	Default Values	Description
S2MM_DATA_RAM_TYPE	LSRAM uSRAM FABRIC	LSRAM	S2MM Data FIFO RAM Type This parameter is used to configure RAM implementation for the data FIFO. This parameter can be configured only when S2MM Data FIFO Enable is selected.
S2MM_PKT_FIFO_ENABLE	Cut Through Store and Forward	Cut Through	S2MM Store and Forward FIFO Enable This parameter is used to select Cut Through mode or Store and Forward mode of data FIFO. When Store and Forward mode is selected, data FIFO is not read until the last beat of AXI4-Stream transfer is received. When Cut Through mode is selected, data FIFO is read immediately that is, when data FIFO is not empty. When Store and Forward mode is selected, S2MM Length register and S2MM Data FIFO Depth must be configured such that the data FIFO must be able to store number of bytes configured in S2MM Length register. For example, if S2MM Data Width is configured to 64 (8 bytes) and S2MM Data FIFO Depth is configured to 2K, which can store 16K bytes then S2MM Length register must not be configured to more than 16K bytes. This parameter can be configured only if S2MM Data FIFO Enable is selected.
S2MM_ENDIAN_CONV	0-1	0	S2MM Enable Big to Little Endian Conversion 0: Disable endian conversion. This option must be selected when data input on the AXI4-Stream interface is in little-endian format. 1: Enable endian conversion. This option must be selected when data input on the AXI4-Stream interface is in big-endian format. Endian conversion is performed at byte level that is, MSB of the AXI4-Stream interface data is placed on LSB of the AXI4-Memory Mapped interface data and similarly other bytes positions are changed. Endian conversion is performed for the AXI4-Stream data and user ports (when S2MM User Enable is selected).
S2MM_CMDSTS_FIFO_ENABLE	0-1	1	S2MM Command/Status FIFO Enable This parameter is used to enable or disable command/status FIFO. When this parameter is enabled, two FIFO, command FIFO, and status FIFO are inferred by the IP Core. Command FIFO stores S2MM transfer request and status FIFO stores S2MM transfer write response. You can push S2MM transfer request into the command FIFO by writing descriptor. One descriptor consists of S2MM Length register, S2MM Address0 register, S2MM Address1 register (if S2MM Address Width is set to more than 32), and S2MM Control register. When start (0th bit) of S2MM Control register is written, descriptor will be stored into command FIFO. Similarly, for each descriptor, write response received from the AXI4-Memory Mapped is stored into status FIFO. IP core stores the Done (0th bit) flag when the number of bytes configured in S2MM Length registers are transmitted to the AXI4-Memory Mapped interface. When S2MM Undefined Burst Length Enable is selected, number of bytes received from the AXI4-Stream interface is also stored in status FIFO. You can read number of bytes received by reading MM2S Length Register . You can read the status FIFO by reading MM2S Status register.

.....continued

Name	Valid Values	Default Values	Description
S2MM_CMDSTS_FIFO_DEPTH	8 16 32 64 128 256 512 1024	16	Number of S2MM transfer request and response can be stored in Command/Status FIFO This parameter is used to select number of S2MM transfer request and response can be stored in command/status FIFO. This parameter can be configured only when S2MM Command/Status FIFO Enable is selected.
S2MM_CMDSTS_RAM_TYPE	LSRAM uSRAM Fabric	uSRAM	S2MM Command/Status FIFO RAM Type This parameter is used to configure RAM implementation for the command/status FIFO. This parameter can be configured only when S2MM Command/Status FIFO Enable is selected.
S2MM_USER_ENABLE	0-1	0	S2MM User Enable This parameter is used to enable/disable the AXI4-Stream and AXI4 interface user ports.
S2MM_USER_WIDTH	1-2048	1	S2MM User Width (in bits) This parameter is used to select width for user ports of the AXI4-Stream target and the AXI4 initiator (write data channel) interface. This parameter can be configured only when S2MM User Enable is enabled. This parameter must be configured in multiple of eight when S2MM Enable Big to Little Endian Conversion is selected.
S2MM_UNDEF_BSTLEN	0-1	0	S2MM Undefined Burst Length Enable This parameter is used to enable or disable undefined burst length as listed: 0: Disable the undefined burst length for S2MM 1: Enables the undefined burst length for S2MM This parameter must be enabled when the number of bytes to be received on the AXI4-Stream interface of S2MM block is unknown.
S2MM_BURST_LENGTH	2 4 8 16 32 64 128 256	32	S2MM Burst Length This parameter is used to defines the maximum AXI4-Memory Mapped burst length. This parameter can be configured only when S2MM Undefined Burst Length Enable is selected.
S2MM_PKT_DROP_OVF	0-1	0	S2MM-Drop the packets when S2MM Data FIFO is overflow. This parameter can be configured only if GUI parameter S2MM Data FIFO Enable is enabled and GUI parameter S2MM Store and Forward FIFO Enable is enabled. When this parameter is enabled, packet will be dropped when S2MM Data FIFO is overflow and overflow bit in S2MM status register will be set to 1.
S2MM_PKT_DROP_ERR	0-1	0	S2MM-Drop the packet when input port S2MM_PKT_ERR is asserted. Packet will be dropped when input port S2MM_PKT_ERR is asserted at end of packet boundary, that is, when T_AXI4S_TLAST of S2MM is asserted.

.....continued

Name	Valid Values	Default Values	Description
RESET_TYPE	Asynchronous Synchronous	Asynchronous	Reset Type This parameter is used to select reset type.

4.2 Inputs and Outputs Signals [\(Ask a Question\)](#)

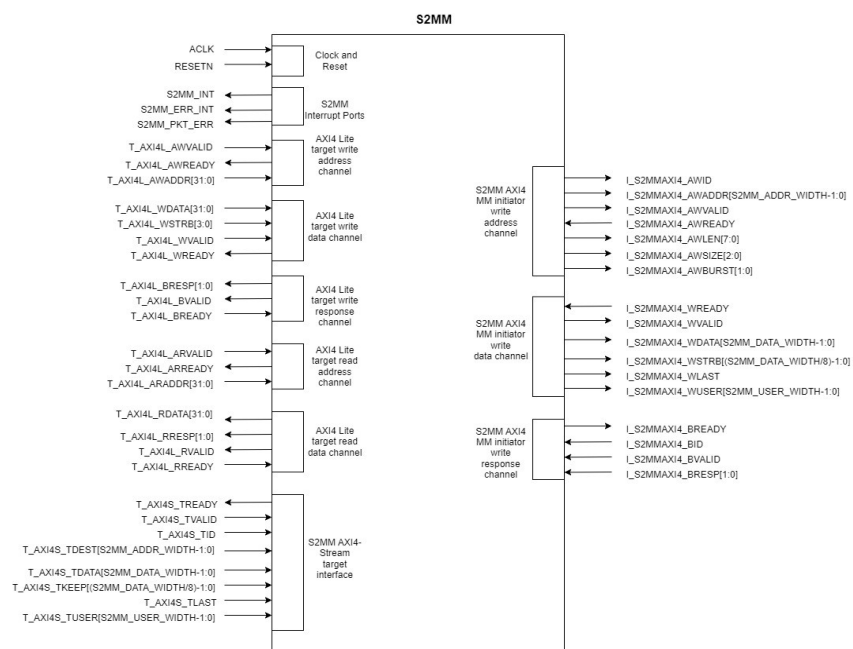
The following table lists the clock ports for CoreAXI4ProtoConv.

Table 4-2. Clock and Reset Ports

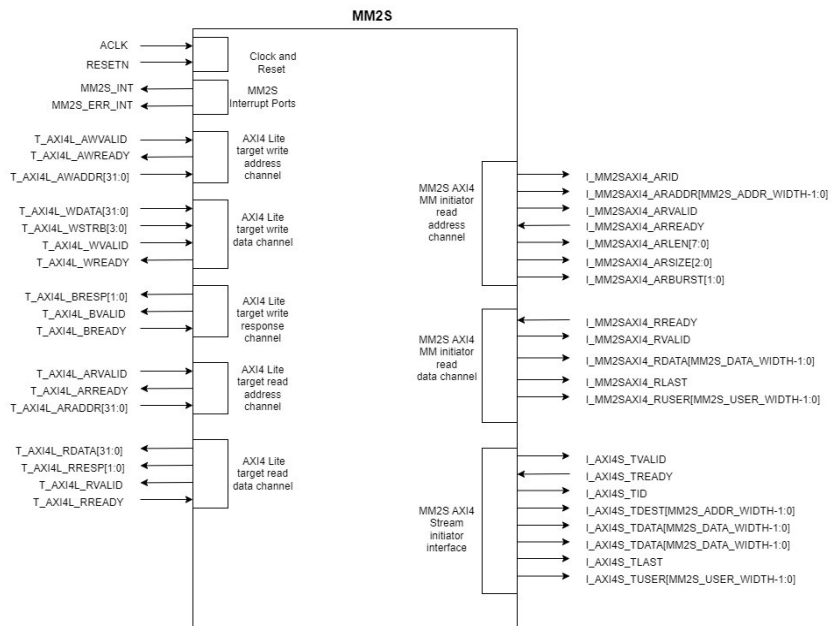
Port Name	Direction	Width	Description
Clock and Reset Ports			
ACLK	Input	1	All the interfaces used into S2MM or MM2S block (AXI4-Lite target, AXI4-Stream target, and AXI4 initiator) uses ACLK.
RESETN	Input	1	This active-low reset resets the core. RESETN must be externally synchronized with the ACLK clock domain. Note: The reset is synchronous or asynchronous based on the RESET_TYPE parameter

The following figure shows the block diagram of S2MM input and output ports.

Figure 4-1. Block Diagram of S2MM Input and Output Ports



The following figure shows the block diagram of MM2S input and output ports.

Figure 4-2. Block Diagram of MM2S Input and Output Ports

The following table lists the input and output ports of S2MM and MM2S.

Table 4-3. Input and Output Ports

Port Name	Direction	Clock Domain	Width	Description
AXI4-Lite Port Interface				
T_AXI4L_AWVALID	Input	ACLK	1	AXI4-Lite write address valid This signal indicates that valid write address and control information are available.
T_AXI4L_AWREADY	Output	ACLK	1	AXI4-Lite write address ready This signal indicates that the target is ready to accept an address and associated control signals. When the RESETN signal is asserted, this signal is asserted. When the AXI4-Lite write address phase is completed, this signal is de-asserted, and when the AXI4-Lite write response phase is completed, this signal is asserted again.
T_AXI4L_AWADDR	Input	ACLK	11	AXI4-Lite write address
T_AXI4L_WDATA	Input	ACLK	32	AXI4-Lite write data
T_AXI4L_WSTRB	Input	ACLK	4	AXI4-Lite write strobes
T_AXI4L_WVALID	Input	ACLK	1	AXI4-Lite write valid
T_AXI4L_WREADY	Output	ACLK	1	AXI4-Lite Write ready
T_AXI4L_BRESP	Output	ACLK	2	AXI4-Lite write response CoreAXI4ProtoConv generates error response (SLVERR) on AXI4-Lite write response channel when register address other than mentioned in register address map (invalid address) is accessed.
T_AXI4L_BVALID	Output	ACLK	1	AXI4-Lite write response valid This signal is asserted when AXI4-Lite data phase is completed and de-asserted when AXI4-Lite write response phase is completed.

.....continued

Port Name	Direction	Clock Domain	Width	Description
T_AXI4L_BREADY	Input	ACLK	1	AXI4-Lite response ready
T_AXI4L_ARADDR	Input	ACLK	11	AXI4-Lite read address The read address gives the address of the first transfer in a read burst transaction.
T_AXI4L_ARVALID	Input	ACLK	1	AXI4-Lite read address valid This signal indicates that the channel is signaling valid read address and control information.
T_AXI4L_ARREADY	Output	ACLK	1	AXI4-Lite response ready This signal indicates that the target is ready to accept an address and associated control signals.
T_AXI4L_RDATA	Output	ACLK	32	AXI4-Lite read data
T_AXI4L_RRESP	Output	ACLK	2	AXI4-Lite read response CoreAXI4ProtoConv generates error response (SLVERR) on AXI4-Lite read response, when register address other than mentioned in register address map (invalid address) is accessed.
T_AXI4L_RVALID	Output	ACLK	1	AXI4-Lite read valid This signal indicates that the channel is signaling the required read data.
T_AXI4L_RREADY	Input	ACLK	1	AXI4-Lite read ready This signal indicates that the initiator can accept the read data and response information.

S2MM Ports**AXI4-Stream Target Port Interface**

T_AXI4S_TREADY	Output	ACLK	1	AXI4-Stream ready indicates that the target can accept a transfer in the current cycle.
T_AXI4S_TVALID	Input	ACLK	1	AXI4-Stream valid indicates that the initiator is driving a valid transfer.
T_AXI4S_TID	Input	ACLK	1	AXI4-Stream TID is the data stream identifier that indicates different streams of data. This port is not used by the IP core.
T_AXI4S_TDEST	Input	ACLK	S2MM_ADDR_WIDTH	AXI4-Stream destination provides routing information for the data stream. This port is not used by the IP core.
T_AXI4S_TDATA	Input	ACLK	S2MM_DATA_WIDTH	AXI4-Stream data is the primary payload that is used to provide the data that is passing across the interface.
T_AXI4S_TKEEP	Input	ACLK	S2MM_DATA_WIDTH / 8	AXI4-Stream byte qualifier that indicates whether the content of the associated byte of data is processed as part of the data stream. Associated bytes that have the TKEEP byte qualifier de-asserted are null bytes and can be removed from the data stream. CoreAXI4ProtoConv does not support null bytes between the packet boundaries. When S2MM Undefined Burst Length Enable is enabled, null bytes is supported when T_AXI4S_TLAST is asserted that is, at the packet boundary.
T_AXI4S_TLAST	Input	ACLK	1	AXI4-Stream last indicates the boundary of a packet.
T_AXI4S_TUSER	Input	ACLK	S2MM_USER_WIDTH	AXI4-Stream target user defined sideband information that can be transmitted alongside the data stream.

AXI4 Initiator Write Interface Ports

.....continued

Port Name	Direction	Clock Domain	Width	Description
I_S2MMAXI4_AWID	Output	ACLK	1	AXI4 write address ID This port is not used by the IP core and always driven to 0.
I_S2MMAXI4_AWADDR	Output	ACLK	S2MM_ADDR_WIDTH	AXI4 initiator write address The write address gives the address of the first transfer in a write burst transaction.
I_S2MMAXI4_AWVALID	Output	ACLK	1	AXI4 initiator write address valid This signal indicates that valid write address and control information are available.
I_S2MMAXI4_AWREADY	Input	ACLK	1	AXI4 initiator write address ready This signal indicates that the target is ready to accept an address and associated control signals.
I_S2MMAXI4_AWLEN	Output	ACLK	8	AXI4 initiator the burst length gives the exact number of transfers in a burst. This information determines the number of data transfers associated with the address.
I_S2MMAXI4_AWSIZE	Output	ACLK	3	AXI4 initiator burst size signal indicates the size of each transfer in the burst.
I_S2MMAXI4_AWBURST	Output	ACLK	2	AXI4 initiator burst type The burst type and the size information, determine how the address for each transfer within the burst is calculated. IP core supports only FIXED and INCR burst type.
I_S2MMAXI4_WREADY	Input	ACLK	1	AXI4 initiator write ready This signal indicates that the target can accept the write data.
I_S2MMAXI4_WVALID	Output	ACLK	1	AXI4 initiator write valid This signal indicates that valid write data and strobes are available.
I_S2MMAXI4_WDATA	Output	ACLK	S2MM_DATA_WIDTH	AXI4 initiator write data
I_S2MMAXI4_WSTRB	Output	ACLK	S2MM_DATA_WIDTH/8	AXI4 initiator write strobes This signal indicates which byte lanes hold valid data. There is one write strobe bit for each eight bits of the write data bus.
I_S2MMAXI4_WLAST	Output	ACLK	1	AXI4 initiator write last This signal indicates the last transfer in a write burst.
I_S2MMAXI4_WUSER	Output	ACLK	S2MM_USER_WIDTH	AXI4 initiator write user signal
I_S2MMAXI4_BID	Input	ACLK	1	AXI4 initiator response ID tag
I_S2MMAXI4_BREADY	Output	ACLK	1	AXI4 initiator response ready This signal indicates that the initiator can accept a write response.
I_S2MMAXI4_BVALID	Input	ACLK	1	AXI4 initiator write response valid This signal indicates the status of the write transaction. IP core samples this signal when signal I_S2MMAXI4_BVALID and signal I_S2MMAXI4_BREADY are asserted and latches the response. You can read the response through S2MM status register.
I_S2MMAXI4_BRESP	Input	ACLK	2	AXI4 initiator write response This signal indicates the status of the write transaction.

S2MM Interrupt Ports

.....continued

Port Name	Direction	Clock Domain	Width	Description
S2MM_INT	Output	ACLK	1	Active-High interrupt signal. It is a level triggered interrupt. The configuration of this interrupt will be facilitated using the S2MM Interrupt Enable Register, and it can be cleared with the S2MM Interrupt Source Register. S2MM Interrupt Source Register can be read when interrupt is asserted. S2MM_INT is asserted when S2MM Done or S2MM Multi PKT Done bits of the S2MM Interrupt Source Register is set to 1 and corresponding bits of S2MM Interrupt Enable register is set to 1.
S2MM_ERR_INT	Output	ACLK	1	Active-High interrupt signal. It is a level triggered interrupt. The configuration of this interrupt will be facilitated using the S2MM Interrupt Enable Register, and it can be cleared with the S2MM Interrupt Source Register. This interrupt is typically used to signal an error condition in the system. When an error occurs, the S2MM_ERR_INT port is asserted. S2MM_ERR_INT is asserted when S2MM PKT DROP OVF, S2MM PKT DROP ERR, and S2MM AXI4 Response Error (either SLVERR or DECERR is 1) of the S2MM Interrupt Source Register is set to 1 and corresponding bits of S2MM Interrupt Enable register is set to 1.
S2MM_PKT_ERR	Input	ACLK	1	Core drops the packet when this port is set to high. This port will be sampled on EOP boundary (AXI4-Stream TLAST) is detected. This port will be exposed to the user only when GUI parameter S2MM Store and Forward FIFO Enable is enabled. Core generates start of packet (SOP) and end of packet boundaries (EOP) from the AXI4 Stream T_AXI4S_TVALID, T_AXI4S_TREADY and T_AXI4S_TLAST ports. Whenever new SOP is generated, write pointer of the S2MM Data FIFO buffer will be latched. When S2MM_PKT_ERR is high on the EOP boundary, core loads the write pointer latched on SOP into the write pointer and stops the buffer write operation. When EOP is detected, core generates an interrupt if enabled to indicate S2MM is completed. In the status register, packet drop bit will be set. This port can be used to drop the packet on the event of error detection in the external source device.

MM2S Ports**AXI4-Stream Initiator Port Interface**

I_AXI4S_TVALID	Output	ACLK	1	AXI4-Stream initiator valid indicates that the initiator is driving a valid transfer. A transfer takes place when both I_AXI4S_TVALID and I_AXI4S_TREADY are asserted.
I_AXI4S_TREADY	Input	ACLK	1	AXI4-Stream initiator ready indicates that the initiator can accept a transfer in the current cycle.
I_AXI4S_TDATA	Output	ACLK	MM2S_DATA_WIDTH	AXI4-Stream initiator data is the primary payload that is used to provide the data that is passing across the interface.
I_AXI4S_TKEEP	Output	ACLK	MM2S_DATA_WIDTH / 8	AXI4-Stream initiator byte qualifier that indicates whether the content of the associated byte of data is processed as part of the data stream.
I_AXI4S_TLAST	Output	ACLK	1	AXI4-Stream initiator last indicates the boundary of a packet.

.....continued

Port Name	Direction	Clock Domain	Width	Description
I_AXI4S_TID	Output	ACLK	1	AXI4-Stream TID is the data stream identifier that indicates different streams of data. This port is not used by the IP core and always driven to 0.
I_AXI4S_TDEST	Output	ACLK	MM2S_ADDR_WIDTH	AXI4-Stream initiator destination provides routing information for the data stream. This port is not used by the IP core and always driven to 0.
I_AXI4S_TUSER	Output	ACLK	MM2S_USER_WIDTH	AXI4-Stream initiator user defined sideband information that can be transmitted alongside the data stream.
AXI4 Initiator Read Interface Ports				
I_MM2SAXI4_ARID	Output	ACLK	1	AXI4 initiator read address ID
I_MM2SAXI4_ARREADY	Input	ACLK	1	AXI4 target read address ready This signal indicates that the target is ready to accept an address and associated control signals.
I_MM2SAXI4_ARVALID	Output	ACLK	1	AXI4 initiator read address valid This signal indicates that the channel is signaling valid read address and control information.
I_MM2SAXI4_ARADDR	Output	ACLK	MM2S_ADDR_WIDTH	AXI4 initiator read address The read address gives the address of the first transfer in a read burst transaction.
I_MM2SAXI4_ARLEN	Output	ACLK	8	AXI4 initiator burst length This signal indicates the exact number of transfers in a burst.
I_MM2SAXI4_ARSIZE	Output	ACLK	3	AXI4 initiator burst size This signal indicates the size of each transfer in the burst.
I_MM2SAXI4_ARBURST	Output	ACLK	2	AXI4 initiator burst type The burst type and the size information determine how the address for each transfer within the burst is calculated. IP core supports only FIXED and INCR burst type.
I_MM2SAXI4_RID	Input	ACLK	1	AXI4 initiator read ID tag. This port is not used by the IP core.
I_MM2SAXI4_RREADY	Output	ACLK	1	AXI4 initiator read ready This signal indicates that the initiator can accept the read data and response information.
I_MM2SAXI4_RVALID	Input	ACLK	1	AXI4 target read valid This signal indicates that the channel is signaling the required read data.
I_MM2SAXI4_RDATA	Input	ACLK	MM2S_DATA_WIDTH	AXI4 target read data
I_MM2SAXI4_RLAST	Input	ACLK	1	AXI4 target read last This signal indicates the last transfer in a read burst.
I_MM2SAXI4_RRESP	Input	ACLK	2	AXI4 read response This signal indicates the status of the read transfer. IP core samples this signal for each read transfer and when OK response is not received for any transfer then latches the error response. You can read the response through MM2S status register.
I_MM2SAXI4_RUSER	Input	ACLK	1	AXI4 target user defined sideband information that can be transmitted alongside the data stream.

.....continued

Port Name	Direction	Clock Domain	Width	Description
MM2S Interrupt Ports				
MM2S_INT	Output	ACLK	1	Active-High interrupt signal. It is a level triggered interrupt. The configuration of this interrupt will be facilitated using the MM2S Interrupt Enable Register, and it can be cleared with the MM2S Interrupt Source Register. MM2S Interrupt Source Register can be read when interrupt is asserted. MM2S_INT is asserted when MM2S Done bit of the MM2S Interrupt Source Register is set to 1 and corresponding bits of MM2S Interrupt Enable register is set to 1.
MM2S_ERR_INT	Output	ACLK	1	Active-High interrupt signal. It is a level triggered interrupt. The configuration of this interrupt will be facilitated using the MM2S Interrupt Enable Register, and it can be cleared with the MM2S Interrupt Source Register. This interrupt is typically used to signal an error condition in the system. When an error occurs, the MM2S_ERR_INT port is asserted. MM2S_ERR_INT is asserted when MM2S AXI4 Response Error (either SLVERR or DECERR) is 1 of the MM2S Interrupt Source Register is set to 1 and corresponding bits of MM2S Interrupt Enable register is set to 1.

**Important:**

- Read address and read data channel of the S2MM AXI4 initiator interface are not used by the IP core.
- Write address, write data, and write response channel of the MM2S AXI4 initiator interface are not used by the IP core.

5. Timing Diagrams [\(Ask a Question\)](#)

This section discusses various timing diagrams.

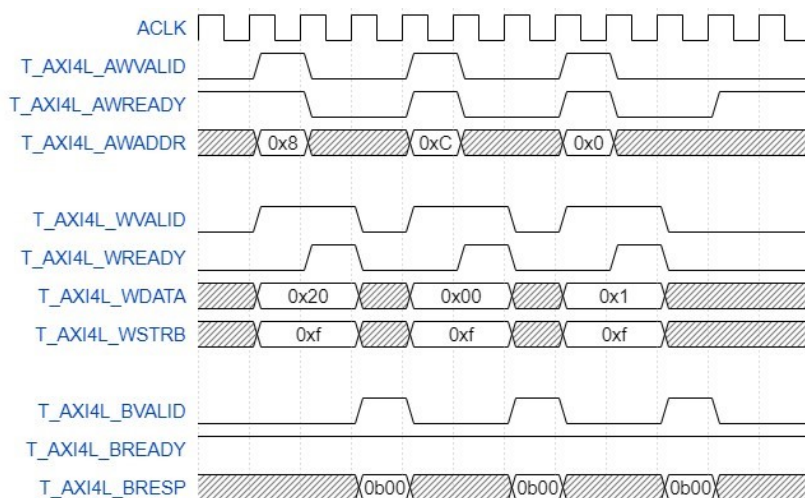
5.1 S2MM Timing Diagrams [\(Ask a Question\)](#)

The section shows the S2MM timing diagrams.

5.1.1 S2MM AXI4 Lite Target Write Transaction [\(Ask a Question\)](#)

The following figure shows the S2MM AXI4 lite target write transaction timing diagram.

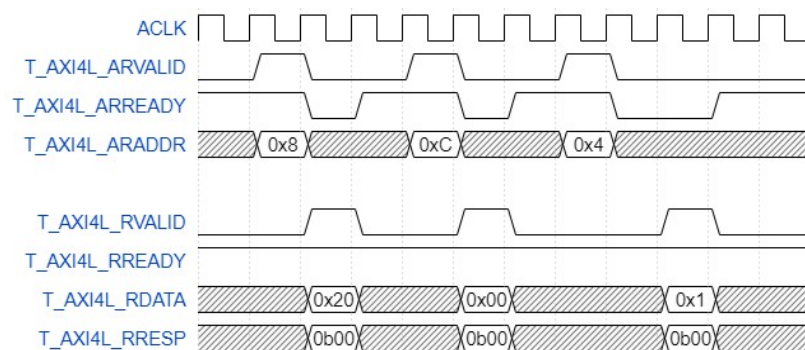
Figure 5-1. AXI4 Lite Target Write Transaction



5.1.2 S2MM AXI4 Lite Target Read Transaction [\(Ask a Question\)](#)

The following figure shows the S2MM AXI4 lite target read transaction timing diagram.

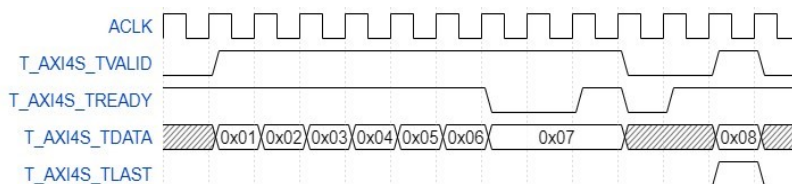
Figure 5-2. AXI4 Lite Target Read Transaction



5.1.3 S2MM AXI4 Stream Target Transaction [\(Ask a Question\)](#)

The following figure shows the S2MM AXI4 stream target transaction timing diagram.

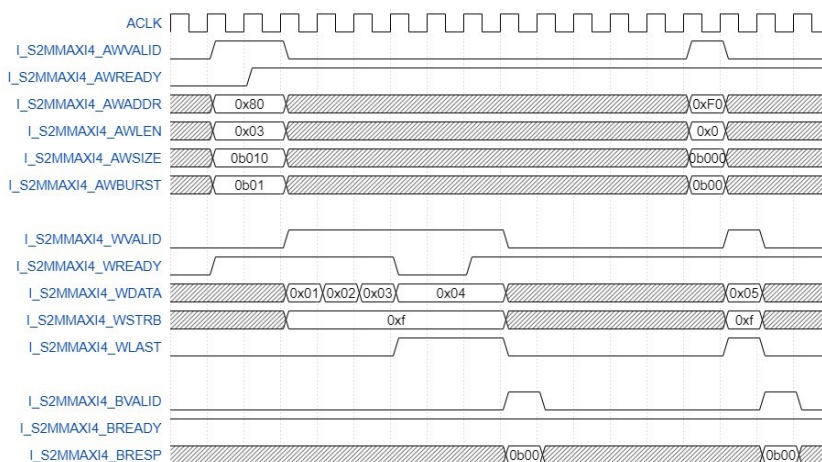
Figure 5-3. AXI4 Stream Target Transaction



5.1.4 S2MM AXI4 Memory Mapped Initiator Write Transaction [\(Ask a Question\)](#)

The following figure shows the S2MM AXI4 memory mapped initiator write transaction timing diagram.

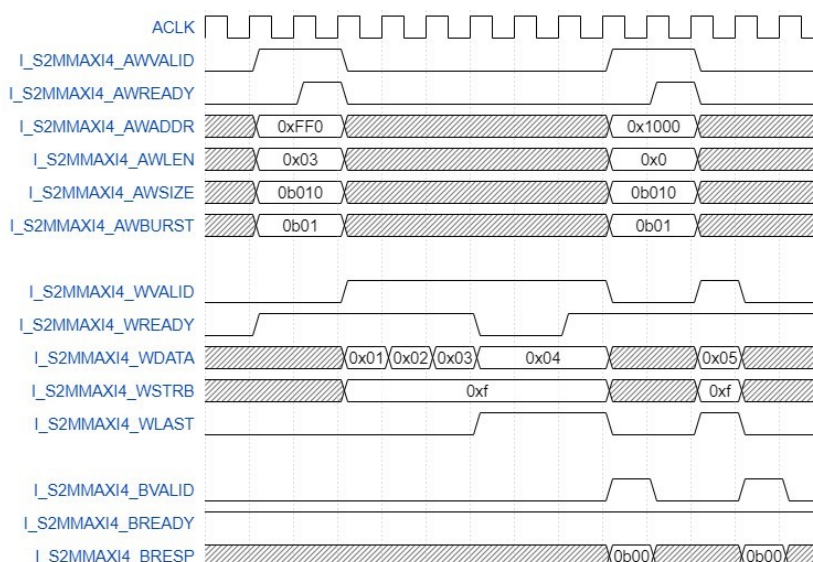
Figure 5-4. AXI4 Memory Mapped Initiator Write Transaction



5.1.5 S2MM AXI4 Memory Mapped Initiator Write Transaction with 4K Address Boundary Crossing

The following figure shows the S2MM AXI4 memory mapped initiator write transaction with 4K address boundary crossing timing diagram.

Figure 5-5. AXI4 Memory Mapped Initiator Write Transaction with 4K Address Boundary Crossing



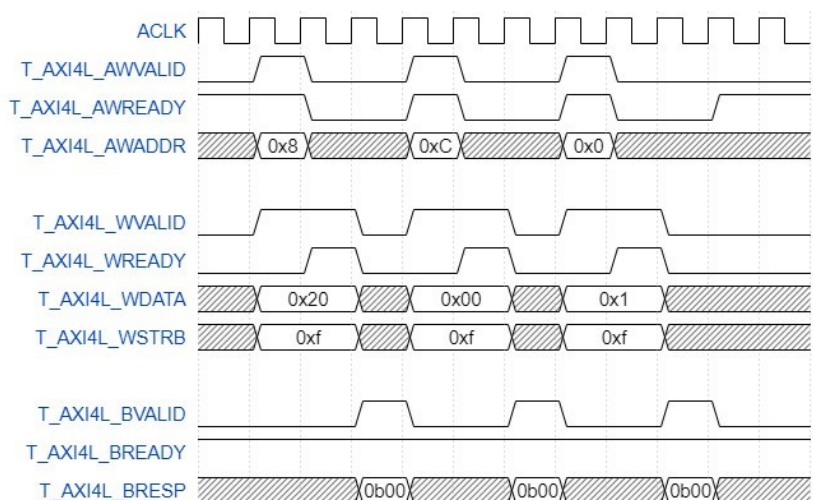
5.2 MM2S Timing Diagrams [\(Ask a Question\)](#)

The section shows the MM2S timing diagrams.

5.2.1 MM2S AXI4 Lite Target Write Transaction [\(Ask a Question\)](#)

The following figure shows the MM2S AXI4 lite target write transaction timing diagram.

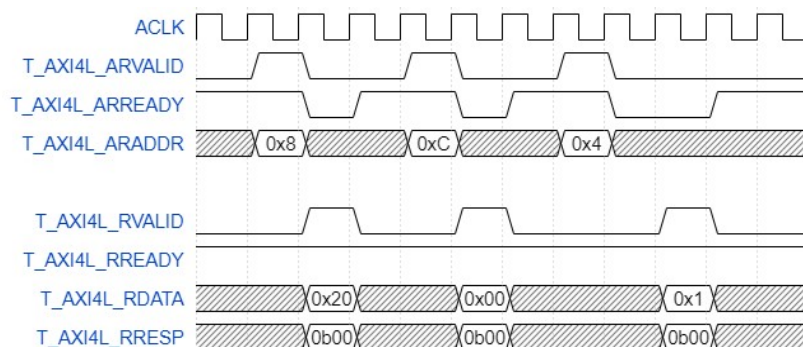
Figure 5-6. AXI4 Lite Target Write Transaction



5.2.2 MM2S AXI4 Lite Target Read Transaction [\(Ask a Question\)](#)

The following figure shows the MM2S AXI4 lite target read transaction timing diagram.

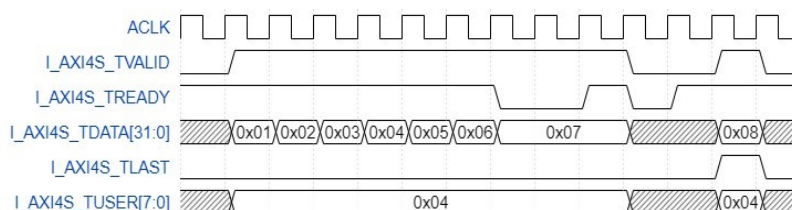
Figure 5-7. AXI4 Lite Target Read Transaction



5.2.3 MM2S AXI4 Stream Initiator Transaction [\(Ask a Question\)](#)

The following figure shows the MM2S AXI4 stream initiator transaction timing diagram.

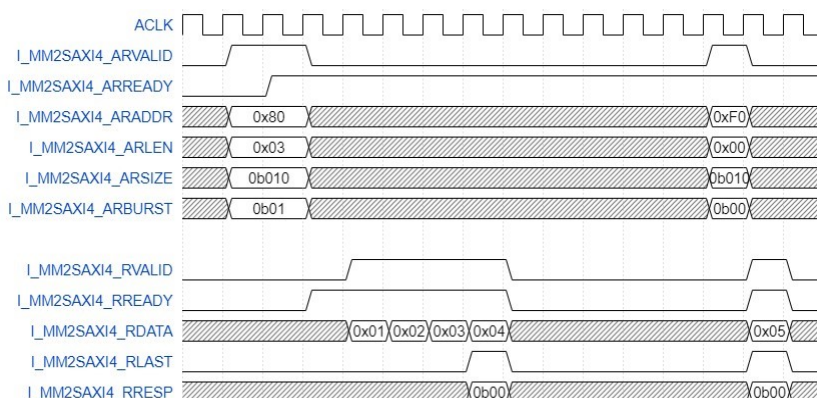
Figure 5-8. AXI4 Stream Initiator Transaction



5.2.4 MM2S AXI4 Memory Mapped Initiator Read Transaction [\(Ask a Question\)](#)

The following figure shows the MM2S AXI4 memory mapped initiator read transaction timing diagram.

Figure 5-9. AXI4 Memory Mapped Initiator Read Transaction

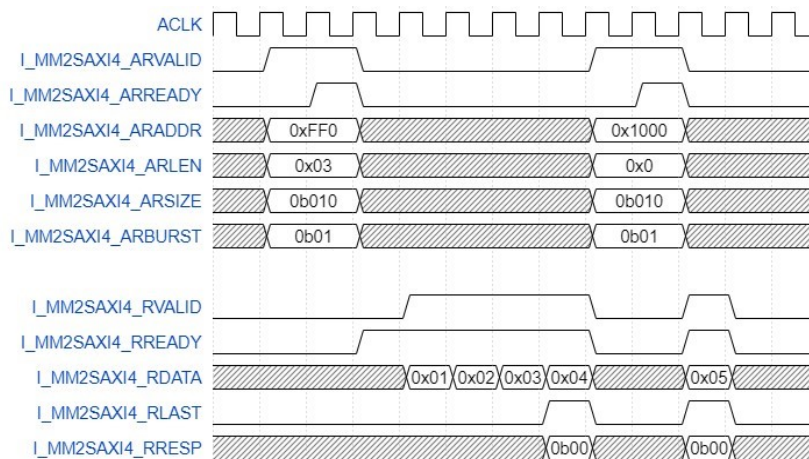


5.2.5 MM2S AXI4 Memory Mapped Initiator Read Transaction with 4K Address Boundary Crossing

Crossing ([Ask a Question](#))

The following figure shows the MM2S AXI4 memory mapped initiator read transaction with 4K address boundary crossing timing diagram.

Figure 5-10. AXI4 Memory Mapped Initiator Read Transaction with 4K Address Boundary Crossing



6. Implementation of CoreAXI4ProtoConv [\(Ask a Question\)](#)

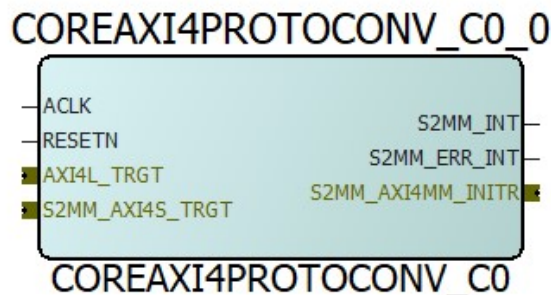
This section describes implementation of the CoreAXI4ProtoConv in Libero Design Suite.

6.1 SmartDesign [\(Ask a Question\)](#)

CoreAXI4ProtoConv is available for download to the SmartDesign IP Catalog through the Libero SOC web repository. For information on using SmartDesign to instantiate, configure, connect, and generate cores, see the [Libero SOC online help](#).

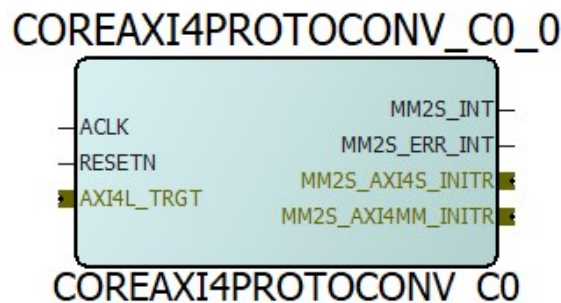
The following figure shows an example of an instantiated view of CoreAXI4ProtoConv S2MM on the SmartDesign.

Figure 6-1. Instantiation of CoreAXI4ProtoConv S2MM on SmartDesign



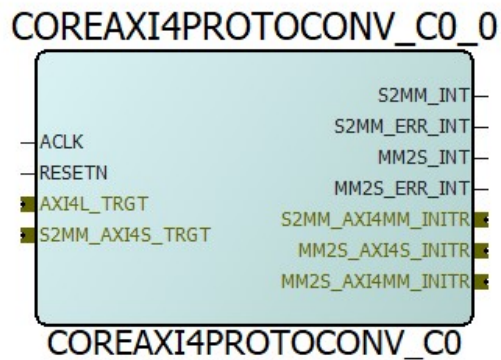
The following figure shows an example of an instantiated view of CoreAXI4ProtoConv MM2S on the SmartDesign.

Figure 6-2. Instantiation of CoreAXI4ProtoConv MM2S on SmartDesign



The following figure shows an example of an instantiated view of CoreAXI4ProtoConv on the SmartDesign.

Figure 6-3. Instantiation of CoreAXI4ProtoConv on SmartDesign

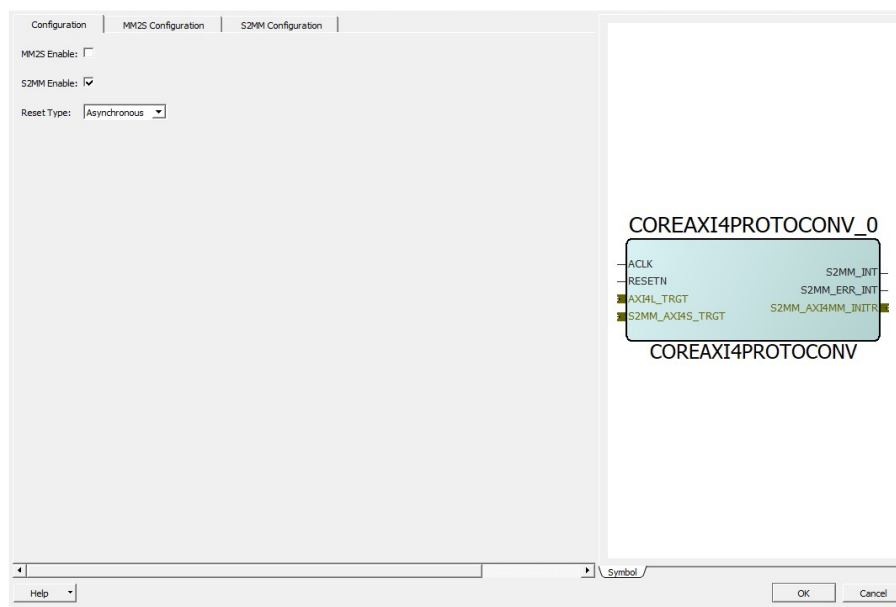


6.1.1 Configuring the CoreAXI4ProtoConv [\(Ask a Question\)](#)

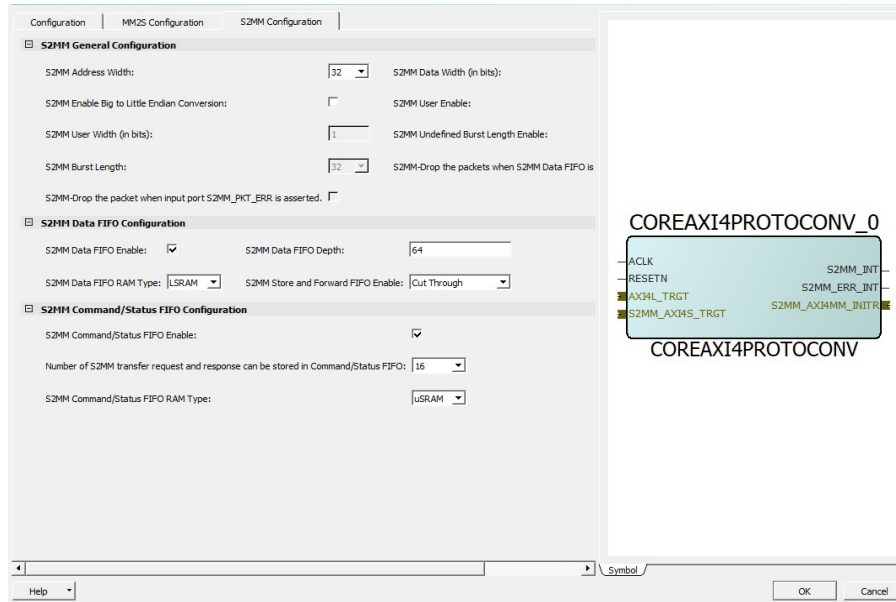
This section shows how the core instance can be configured using its configuration GUI.

The following figure shows the Enabling Block of S2MM.

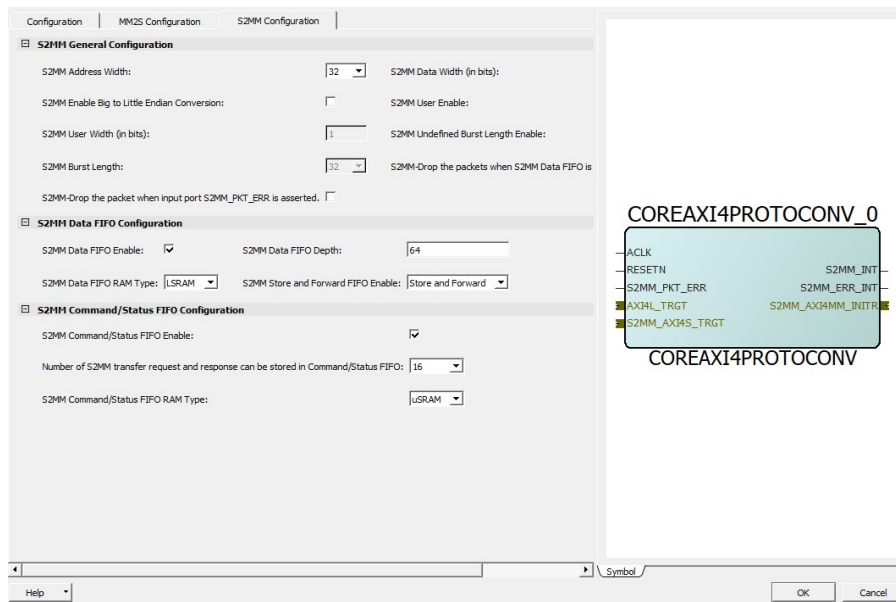
Figure 6-4. CoreAXI4ProtoConv Configuration GUI (Enabling Block of S2MM)



The following figure shows the Configuration tab of S2MM.

Figure 6-5. CoreAXI4ProtoConv Configuration GUI (Configuration Tab of S2MM)

The following figures shows the Configuration tab of S2MM when S2MM Store and Forward FIFO Enable is selected.

Figure 6-6. CoreAXI4ProtoConv Configuration GUI (Configuration Tab of S2MM when S2MM Store and Forward FIFO Enable)

6.1.2 Simulation Flow [\(Ask a Question\)](#)

To simulate the core using the testbench, after configuring the CoreAXI4ProtoConv as per the requirement, perform the following steps:

1. Set the design root to the CoreAXI4ProtoConv instantiation in the Libero[®] SoC Design Hierarchy pane, and then click **Build Hierarchy**.
2. Click the **Stimulus Hierarchy** tab, and then click **Build Hierarchy**.

3. Right-click testbench, under **Simulate Pre-Synthesis Design**, select **Open Interactively**.
4. For complete visibility of CoreAXI4ProtoConv input and output ports, add an additional vsim command. To add command, perform the following steps:
 - a. Navigate to **Project > Project Settings > Simulation options > Vsim Commands**.
 - b. Under **Additional options**, add `"-voptargs=+acc"`.
 - c. Click **Save Changes**.

QuestaSim opens with the testbench file and automatically runs the simulation.

 **Important:** If the simulation is interrupted due to the runtime limit specified in the .do file, use the `run-all` command to complete the simulation.

6.1.3 Synthesis in Libero SoC [\(Ask a Question\)](#)

To run synthesis, perform the following steps:

1. With the configuration selected in the configuration GUI, set the design root appropriately.
2. Under **Implement Design**, on the **Design Flow** tab, right-click **Synthesize** and click **Run**.

6.1.4 Place-and-Route in Libero SoC [\(Ask a Question\)](#)

To run the place and route, perform the following steps:

1. With the configuration selected in the configuration GUI, set the design root appropriately.
2. Under **Implement Design**, on the **Design Flow** tab, right-click **Place and Route** and click **Run**.

6.2 Verification [\(Ask a Question\)](#)

This section describes verification of the CoreAXI4ProtoConv.

6.2.1 Testbench [\(Ask a Question\)](#)

A unified testbench is used to verify and test CoreAXI4ProtoConv called as user testbench.

6.2.1.1 User Testbench [\(Ask a Question\)](#)

The following figure shows the user simulation testbench and it includes the instantiation of the CoreAXI4ProtoConv macro, the AXI4-Stream Initiator Model, the AXI4-Stream Target Model, and the AXI4-Memory Mapped Target Model. Testbench also instantiates S2MM Checker and MM2S Checker blocks.

You can find all the testbench environment files from the path - `<project_directory>/component/Actel/DirectCore/CoreAXI4ProtoConv/IP Core Version Number (ex 2.0.100)>/rtl/vlog/test/user`.

AXI4-Stream Initiator Model

This block generates the AXI4-Stream transaction for S2MM. It controls TVALID and TLAST ports of the AXI4-Stream interface. It reads data from the buffer (random data is stored into buffer) and provides to TDATA ports of the AXI4-Stream interface.

AXI4-Stream Target Model

This block controls and randomize the TREADY ports of the AXI4-Stream interface for MM2S. This block is implemented in `axi4s_trgt_model.v` module

AXI4-Memory Mapped Target Model

This block implements the AXI4-Memory Mapped target interface functionality. For S2MM operation, it controls write channel of the AXI4-Memory Mapped interface. It randomize AWREADY and WREADY ports of the AXI4-Memroy Mapped interface. It drives BVALID and BRESP ports when the

last transaction is received. For MM2S operation, it controls read channel of the AXI4-Memory Mapped interface. It randomize AREADY and RVALID ports of the AXI4-Memory Mapped interface. It reads data from the buffer (random data stored into buffer) and provides to RDATA port of the AXI4-Memory Mapped interface. It also generates RLAST at the end of the burst. This block is implemented in `axi4mm_trgt_model.v` module.

S2MM Checker

This block verifies the data sent from the AXI4-Stream interface and received on the AXI4-Memory Mapped interface. It receives data from the AXI4-Lite, the AXI4-Stream, and the AXI4-Memory Mapped (write) interface. It stores the descriptor received from the AXI4-Lite interface into its internal buffer and when buffer is not empty it reads data from the buffer and decodes the S2MM request and derives the start address, burst length and burst type. When it receives the write request on the AXI4-Memory Mapped interface, it compares the start address, burst length and burst type and reports an error if mismatch found. Similarly, it will reads data from the internal buffer (data stored for AXI4-Stream transaction) and compares it when write data is received on the AXI4-Memory Mapped interface and reports an error if data mismatch found. This block is implemented in `s2mm_chkr.v` module.

MM2S Checker

This block verifies the data sent from the AXI4-Memory Mapped interface and received on the AXI4-Stream interface. It receives data from the AXI4-Lite, the AXI4-Stream, and the AXI4-Memory Mapped (read) interface. It stores the descriptor received from the AXI4-Lite interface into its internal buffer and when buffer is not empty it reads data from the buffer and decodes the MM2S request and derives the start address, burst length and burst type. When it receives the read request on the AXI4-Memory Mapped interface, it compares the start address, burst length and burst type and reports an error if mismatch found. Similarly, it will store data into its internal buffer when data is received on the AXI4-Memory Mapped interface. It reads this buffer when data is received on the AXI4-Stream interface and compares with the AXI4-Stream data and reports an error if data mismatch found. This block is implemented in `mm2s_chkr.v` module.

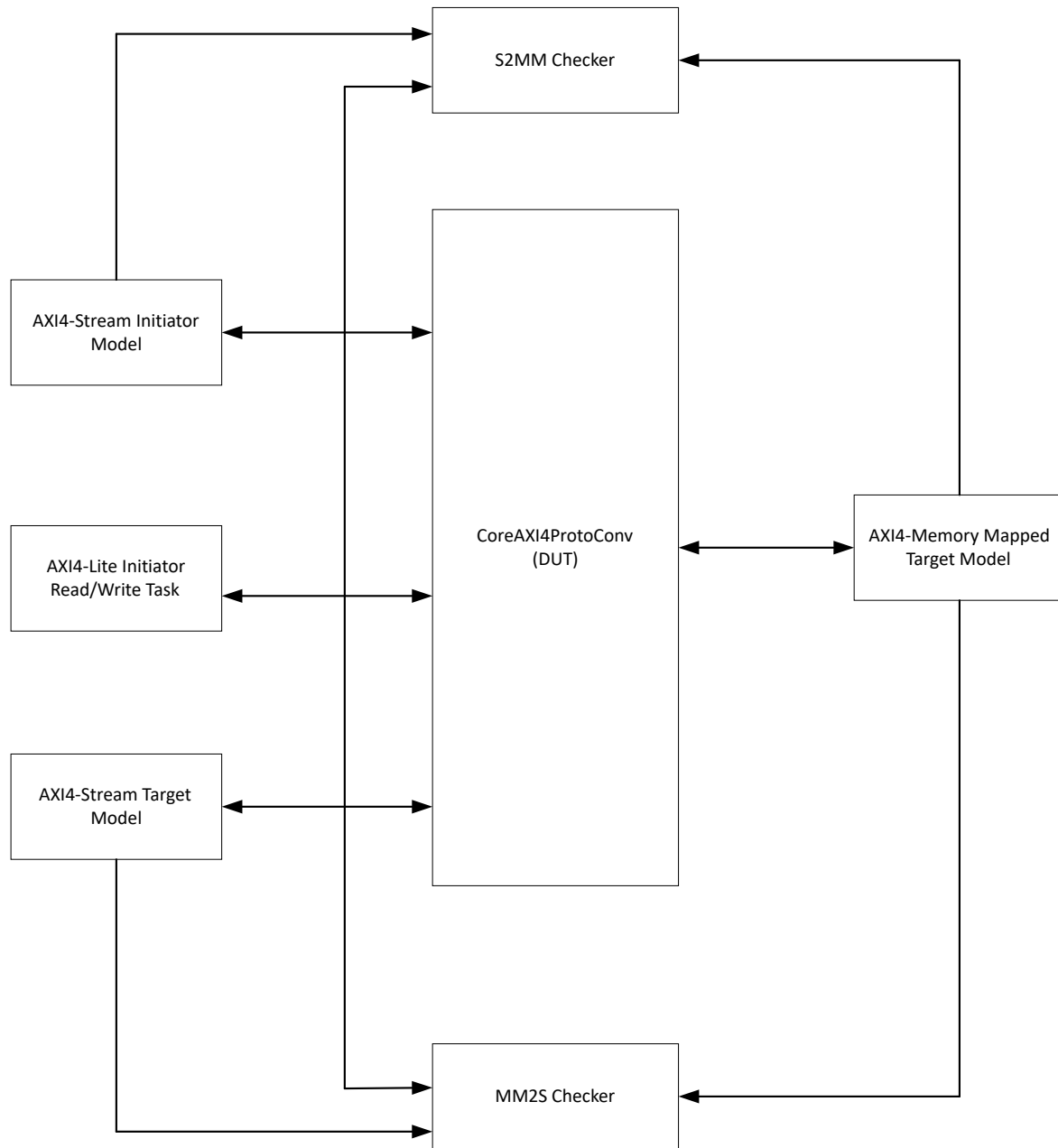
AXI4-Lite Initiator Read/Write Tasks

The AXI4-Lite initiator read/write tasks are implemented in `testbench.v` module. For S2MM/MM2S, `axi4lite_write`, and `axi4lite_read` task are implemented for write and read operation respectively.

Buffer

Internal buffer is used to store the data to send through the AXI4-Stream interface for S2MM and to send through the AXI4-Memory Mapped (read) interface for MM2S. Buffer is instantiated in `testbench.v` file and implemented in `buffer_mem.v` module. This block is implemented in `axi4s_initr_model.v` module.

Figure 6-7. CoreAXI4ProtoConv User Testbench



User testbench supports default configuration only. Following tasks are executed in the user testbench:

- **s2mm_register_rdwr_test:** This task checks the S2MM registers write and read functionality
- **s2mm_test:** This task checks the AXI4-Memory Mapped write channel increment and fixed burst functionality by pushing multiple commands into the command FIFO. This task also used to check the AXI4-Memory Mapped write channel functionality for both increment and fixed burst when 4K address boundary is crossed.
- **s2mm_intr_test:** This task checks the S2MM interrupt functionality.

- **mm2S_register_rdwr_test:** This task checks the MM2S registers write and read functionality
- **mm2s_test:** This task checks the AXI4-Memory Mapped read channel increment and fixed burst functionality by pushing multiple commands into the command FIFO. This task also used to check the AXI4-Memory Mapped read channel functionality for both increment and fixed burst when 4K address boundary is crossed.
- **mm2s_intr_test:** This task checks the MM2S interrupt functionality.

7. S2MM Register Map and Descriptions [\(Ask a Question\)](#)

This section provides the S2MM register descriptions of the CoreAXI4ProtoConv.

Offset	Name	Bit Pos.	7	6	5	4	3	2	1	0
0x00	S2MM_VERSN	7:0								
		15:8								
		23:16	MINOR[7:0]							
		31:24	MAJOR[7:0]							
0x04 ... 0x0F	Reserved									
0x10	S2MM_CTRL	7:0						BURYP[1:0]	START	
		15:8								
		23:16	CMDID[7:0]							
		31:24	CMDID[9:8]							
0x14	S2MM_STS	7:0			PKTDROP_ER R	PKTDROP_OV F	AXI4WRSP[1:0]			DONE
		15:8								
		23:16	CMDID[7:0]							
		31:24	CMDID[9:8]							
0x18	S2MM_LEN	7:0	S2MM_LEN[7:0]							
		15:8	S2MM_LEN[15:8]							
		23:16	S2MM_LEN[23:16]							
		31:24	S2MM_LEN[31:24]							
0x1C	S2MM_ADDR0	7:0	S2MM_ADDR0[7:0]							
		15:8	S2MM_ADDR0[15:8]							
		23:16	S2MM_ADDR0[23:16]							
		31:24	S2MM_ADDR0[31:24]							
0x20	S2MM_ADDR1	7:0	S2MM_ADDR1[7:0]							
		15:8	S2MM_ADDR1[15:8]							
		23:16	S2MM_ADDR1[23:16]							
		31:24	S2MM_ADDR1[31:24]							
0x24	S2MM_INTRENB	7:0				S2MM_IE_PKT DROP_ERR	S2MM_IE_PKT DROP_OVF	S2MM_IE_AXI 4ERR		S2MM_IE_DO NE
		15:8								
		23:16								
		31:24								
0x28	S2MM_INTRSRC	7:0				S2MM_IS_PKT DROP_ERR	S2MM_IS_PKT DROP_OVF	S2MM_IS_AXI 4ERR		S2MM_IS_DO NE
		15:8								
		23:16	S2MM_IS_CMDID[7:0]							
		31:24	S2MM_IS_CMDID[9:8]							
0x2C ... 0xFF	Reserved									
0x0100	S2MM_AXI4BUSERR_CNTR	7:0	S2MM_AXI4BUSERR_CNTR[7:0]							
		15:8	S2MM_AXI4BUSERR_CNTR[15:8]							
		23:16								
		31:24								
0x0104	S2MM_PKTDRPERR_CNTR	7:0	S2MM_PKTDRPERR_CNTR[7:0]							
		15:8	S2MM_PKTDRPERR_CNTR[15:8]							
		23:16								
		31:24								
0x0108	S2MM_PKTDRPOVF_CNTR	7:0	S2MM_PKTDRPOVF_CNTR[7:0]							
		15:8	S2MM_PKTDRPOVF_CNTR[15:8]							
		23:16								
		31:24								

7.1 S2MM_VERSN [\(Ask a Question\)](#)

Name: S2MM_VERSN
Offset: 0x000
Reset: 0x03000000
Property: Read-only

Version register. It will provide Major version and Minor version of the core.

Bit	31	30	29	28	27	26	25	24
	MAJOR[7:0]							
Access	R	R	R	R	R	R	R	R
Reset	0	0	0	0	0	0	1	1
Bit	23	22	21	20	19	18	17	16
	MINOR[7:0]							
Access	R	R	R	R	R	R	R	R
Reset	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8
Access								
Reset								
Bit	7	6	5	4	3	2	1	0
Access								
Reset								

Bits 31:24 – MAJOR[7:0] Major version of the core

Bits 23:16 – MINOR[7:0] Minor version of the core

7.2 S2MM_STS [\(Ask a Question\)](#)

Name: S2MM_STS
Offset: 0x014
Reset: 0x1
Property: Read-only

This register contains status of the S2MM data transfers.

Bit	31	30	29	28	27	26	25	24
							CMDID[9:8]	
Access							R	R
Reset							0	0

Bit	23	22	21	20	19	18	17	16
	CMDID[7:0]							
Access	R	R	R	R	R	R	R	R
Reset	0	0	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8
Access								
Reset								

Bit	7	6	5	4	3	2	1	0
			PKTDROP_ER	PKTDROP_OV	AXI4WRSP[1:0]			DONE
Access			R	F	R	R		R
Reset			0	0	0	0		1

Bits 25:16 – CMDID[9:0] Command ID. Provides command ID for which S2MM transfer is completed. Command ID is not valid when packet drop or overflow error flag is asserted.

Bit 5 – PKTDROP_ERR

Value	Description
0b0	Indicates S2MM packets not dropped due to error
0b1	Indicates S2MM packets dropped due to error

Bit 4 – PKTDROP_OVF

Value	Description
0b0	Indicates S2MM packets not dropped due to overflow
0b1	Indicates S2MM packets dropped due to overflow

Bits 3:2 – AXI4WRSP[1:0]

Value	Description
2b00	OK
2b01	EXOKAY
2b10	SLVERR
2b11	DECERR

Bit 0 – DONE Done.

Value	Description
0b0	Indicates S2MM transfer is not completed
0b1	Indicates S2MM transfer is completed

7.3 S2MM_PKTDRPOVF_CNTR [\(Ask a Question\)](#)

Name: S2MM_PKTDRPOVF_CNTR
Offset: 0x108
Reset: 0x0
Property: Read/Write

This register provides S2MM packet drop count due to S2MM data FIFO overflow.

Bit	31	30	29	28	27	26	25	24
Access								
Reset								
Bit	23	22	21	20	19	18	17	16
Access								
Reset								
Bit	15	14	13	12	11	10	9	8
	S2MM_PKTDRPOVF_CNTR[15:8]							
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0
Bit	7	6	5	4	3	2	1	0
	S2MM_PKTDRPOVF_CNTR[7:0]							
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0

Bits 15:0 – S2MM_PKTDRPOVF_CNTR[15:0] This register provides S2MM packet drop count due to S2MM data FIFO overflow. It is used to count the number of packets is dropped when S2MM Data FIFO is overflow. This register should be accessed only when S2MM_PKT_DROP_OVF parameter is enabled.

7.4 S2MM_PKTDRPERR_CNTR [\(Ask a Question\)](#)

Name: S2MM_PKTDRPERR_CNTR
Offset: 0x104
Reset: 0x0
Property: Read/Write

This register provides S2MM packet drop count due to error detection.

Bit	31	30	29	28	27	26	25	24
Access								
Reset								
Bit	23	22	21	20	19	18	17	16
Access								
Reset								
Bit	15	14	13	12	11	10	9	8
	S2MM_PKTDRPERR_CNTR[15:8]							
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0
Bit	7	6	5	4	3	2	1	0
	S2MM_PKTDRPERR_CNTR[7:0]							
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0

Bits 15:0 – S2MM_PKTDRPERR_CNTR[15:0] This register provides S2MM packet drop count due to error detection. It is used to count the number of packets is dropped when input port S2MM_PKT_ERR is asserted at end of packet boundary, that is, when T_AXI4S_TLAST of S2MM is asserted. This register should be accessed only when S2MM_PKT_DROP_ERR parameter is enabled.

7.5 S2MM_LEN [\(Ask a Question\)](#)

Name: S2MM_LEN
Offset: 0x018
Reset: 0x1
Property: Read/Write

This register contains S2MM length.

Bit	31	30	29	28	27	26	25	24
	S2MM_LEN[31:24]							
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0
Bit	23	22	21	20	19	18	17	16
	S2MM_LEN[23:16]							
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8
	S2MM_LEN[15:8]							
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0
Bit	7	6	5	4	3	2	1	0
	S2MM_LEN[7:0]							
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	1

Bits 31:0 – S2MM_LEN[31:0] Indicates the total number of bytes that are transmitted from the AXI4-Stream interface to the AXI4-Memory Mapped interface. Transfers can be from 1 to $(2^{32}) - 1$ bytes. You must not program "0" value to this register. When S2MM Command/Status FIFO Enable is not enabled, do not write in this register until you complete the current transfer. You can poll the S2MM Done bit of S2MM Status register to know the status of current transfer. When S2MM Undefined Burst Length Enable is enabled, do not program this register. Reading this register when S2MM Undefined Burst Length Enable is enabled, provides number of bytes received in undefined burst transaction.

7.6 S2MM_INTRSRC [\(Ask a Question\)](#)

Name: S2MM_INTRSRC
Offset: 0x028
Reset: 0x0
Property: Read/Write

Interrupt source register

Bit	31	30	29	28	27	26	25	24
							S2MM_IS_CMDID[9:8]	
Access							R/W	R/W
Reset							0	0

Bit	23	22	21	20	19	18	17	16
	S2MM_IS_CMDID[7:0]							
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8
Access								
Reset								

Bit	7	6	5	4	3	2	1	0
				S2MM_IS_PKT DROP_ERR	S2MM_IS_PKT DROP_OVF	S2MM_IS_AXI 4ERR		S2MM_IS_DO NE
Access				R/W	R/W	R/W		R/W
Reset				0	0	0		0

Bits 25:16 – S2MM_IS_CMDID[9:0] Command ID. Provides command ID for which S2MM transfer is completed

Bit 4 – S2MM_IS_PKTDROP_ERR

Value	Description
0b0	Indicates S2MM packets not dropped due to error. Writing 0 to this bit will have no effect
0b1	Indicates S2MM packets dropped due to error. Writing 1 to this bit will clear the bit

Bit 3 – S2MM_IS_PKTDROP_OVF

Value	Description
0b0	Indicates S2MM packets not dropped due to overflow. Writing 0 to this bit will have no effect
0b1	Indicates S2MM packets dropped due to overflow. Writing 1 to this bit will clear the bit

Bit 2 – S2MM_IS_AXI4ERR

Value	Description
0b0	Indicates S2MM AXI4 Error is not detected. Writing 0 to this bit will have no effect
0b1	Indicates S2MM AXI4 Error is detected. Writing 1 to this bit will clear the bit

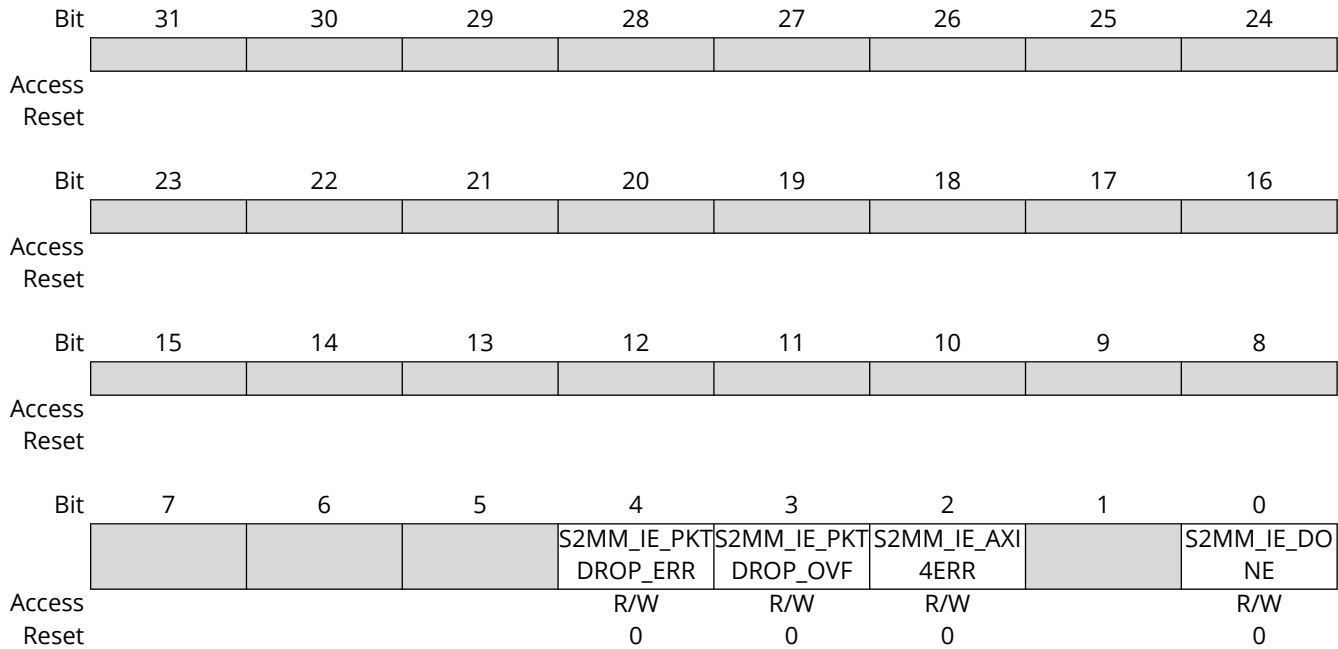
Bit 0 – S2MM_IS_DONE

Value	Description
0b0	Indicates S2MM transfer is not completed. Writing 0 to this bit will have no effect
0b1	Indicates S2MM transfer is completed. Writing 1 to this bit will clear the bit

7.7 S2MM_INTRENB [\(Ask a Question\)](#)

Name: S2MM_INTRENB
Offset: 0x024
Reset: 0x0
Property: Read/Write

Interrupt enable register



Bit 4 – S2MM_IE_PKTDROP_ERR

Value	Description
0b0	Disable interrupt generation when Packet is dropped due to error detection in the received packet.
0b1	Enable interrupt generation when Packet is dropped due to error detection in the received packet.

Bit 3 – S2MM_IE_PKTDROP_OVF

Value	Description
0b0	Disable interrupt generation when Packet is dropped due to fifo overflow
0b1	Enable interrupt generation when Packet is dropped due to fifo overflow

Bit 2 – S2MM_IE_AXI4ERR

Value	Description
0b0	Disable interrupt generation when AXI4 bus error is detected.
0b1	Enable interrupt generation when AXI4 bus error is detected

Bit 0 – S2MM_IE_DONE

Value	Description
0b0	Disable interrupt generation when S2MM done is asserted
0b1	Enable interrupt generation when S2MM done is asserted

7.8 S2MM_CTRL [\(Ask a Question\)](#)

Name: S2MM_CTRL
Offset: 0x010
Reset: 0x0
Property: Read/Write

S2MM control register. This register must be written after writing S2MM length, S2MM address0, and S2MM address1 (if GUI parameter S2MM Address Width is configured to more than 32) registers. When GUI parameter S2MM Command/Status FIFO Enable is not enabled, this register must not be written until the current transfer is completed. User can poll Done bit of S2MM status register to know the status of current transfer

Bit	31	30	29	28	27	26	25	24
							CMDID[9:8]	
Access							R/W	R/W
Reset							0	0
Bit	23	22	21	20	19	18	17	16
	CMDID[7:0]							
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8
Access								
Reset								
Bit	7	6	5	4	3	2	1	0
						BURTP[1:0]		START
Access						R/W	R/W	R/W
Reset						0	0	0

Bits 25:16 – CMDID[9:0] User can assign an arbitrary value to this field. On completion of the S2MM transfer, IP core inserts this field into the S2MM status register. IP core do not support out of order operation. Command issued in the order must be reflected in the same order in the status

Bits 2:1 – BURTP[1:0] If the AXI4 length is "1", FIXED and INCR bursts are equivalent. Fixed burst transfer should be used to access peripheral device which supports fixed address like FIFO. Increment burst transfer should be used to access peripheral device which supports incremental address like SRAM.

Value	Description
2b00	AXI4-Memory Mapped fixed burst
2b01	AXI4-Memory Mapped increment burst
2b10	Reserved
2b11	Reserved

Bit 0 – START When S2MM Command/Status FIFO Enable is enabled, CoreAXI4ProtoConv stores the descriptor into command FIFO. Descriptors consist of S2MM Length, S2MM Address0, S2MM Address1, and S2MM Control register (bits 16 to 25) information. S2MM Start bit is a self-clearing bit.

Value	Description
0b0	There is no impact on S2MM data transfer
0b1	Initiates S2MM data transfer

7.9 S2MM_AXI4BUSERR_CNTR [\(Ask a Question\)](#)

Name: S2MM_AXI4BUSERR_CNTR
Offset: 0x100
Reset: 0x0
Property: Read/Write

This register provides S2MM AXI4 bus error count.

Bit	31	30	29	28	27	26	25	24
Access								
Reset								
Bit	23	22	21	20	19	18	17	16
Access								
Reset								
Bit	15	14	13	12	11	10	9	8
	S2MM_AXI4BUSERR_CNTR[15:8]							
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0
Bit	7	6	5	4	3	2	1	0
	S2MM_AXI4BUSERR_CNTR[7:0]							
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0

Bits 15:0 – S2MM_AXI4BUSERR_CNTR[15:0] This register provides S2MM AXI4 bus error count. S2MM AXI4 bus error counter is incremented when S2MM AXI4WRSP (either SLVERR or DECERR) of S2MM status register is set to 1.

7.10 S2MM_ADDR1 [\(Ask a Question\)](#)

Name: S2MM_ADDR1
Offset: 0x020
Reset: 0x0
Property: Read/Write

This register contains upper 32 bits of the start address of the S2MM data transfer

Bit	31	30	29	28	27	26	25	24
	S2MM_ADDR1[31:24]							
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0
Bit	23	22	21	20	19	18	17	16
	S2MM_ADDR1[23:16]							
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8
	S2MM_ADDR1[15:8]							
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0
Bit	7	6	5	4	3	2	1	0
	S2MM_ADDR1[7:0]							
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0

Bits 31:0 – S2MM_ADDR1[31:0] S2MM address 1. This register is used to configured upper 32 bits of start address for AXI4-Memory Mapped interface. This register must be programmed only when GUI parameter S2MM Address Width configured to more than 32

7.11 S2MM_ADDR0 [\(Ask a Question\)](#)

Name: S2MM_ADDR0
Offset: 0x01C
Reset: 0x0
Property: Read/Write

This register contains lower 32 bits of the start address of the S2MM data transfer

Bit	31	30	29	28	27	26	25	24
	S2MM_ADDR0[31:24]							
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0
Bit	23	22	21	20	19	18	17	16
	S2MM_ADDR0[23:16]							
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8
	S2MM_ADDR0[15:8]							
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0
Bit	7	6	5	4	3	2	1	0
	S2MM_ADDR0[7:0]							
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0

Bits 31:0 – S2MM_ADDR0[31:0] S2MM address 0. This register is used to configured lower 32 bits address for AXI4-Memory Mapped interface. When GUI parameter S2MM Command/Status FIFO Enable is not enabled, this register must not be written until the current transfer is completed. User can poll Done bit of S2MM status register to know the status of current transfer S2MM address register0 and S2MM address register1 (only if GUI parameter S2MM Address Width is configured to more than 32) are used as start address to transfer total number of bytes configured in S2MM length register. If start address and total number of bytes crosses 4K address boundary or burst length is more than 256 then core breaks the S2MM transfer into multiple transfers internally. The CoreAXI4ProtoConv does not support unaligned transfers so start address must always be aligned with the S2MM Data Width.

8. MM2S Register Map and Descriptions [\(Ask a Question\)](#)

This section provides the MM2S register descriptions of the CoreAXI4ProtoConv.

Offset	Name	Bit Pos.	7	6	5	4	3	2	1	0
0x00 ... 0x03FF	Reserved									
0x0400	MM2S_VERSN	7:0								
		15:8								
		23:16	MINOR[7:0]							
		31:24	MAJOR[7:0]							
0x0404 ... 0x040F	Reserved									
0x0410	MM2S_CTRL	7:0						BURYP[1:0]	START	
		15:8								
		23:16	CMDID[7:0]							
		31:24	CMDID[9:8]							
0x0414	MM2S_STS	7:0					AXI4RRSP[1:0]		DONE	
		15:8								
		23:16	CMDID[7:0]							
		31:24	CMDID[9:8]							
0x0418	MM2S_LEN	7:0	MM2S_LEN[7:0]							
		15:8	MM2S_LEN[15:8]							
		23:16	MM2S_LEN[23:16]							
		31:24	MM2S_LEN[31:24]							
0x041C	MM2S_ADDR0	7:0	MM2S_ADDR0[7:0]							
		15:8	MM2S_ADDR0[15:8]							
		23:16	MM2S_ADDR0[23:16]							
		31:24	MM2S_ADDR0[31:24]							
0x0420	MM2S_ADDR1	7:0	MM2S_ADDR1[7:0]							
		15:8	MM2S_ADDR1[15:8]							
		23:16	MM2S_ADDR1[23:16]							
		31:24	MM2S_ADDR1[31:24]							
0x0424	MM2S_INTRENB	7:0						MM2S_IE_AXI 4ERR		MM2S_IE_DO NE
		15:8								
		23:16								
		31:24								
0x0428	MM2S_INTRSRC	7:0						MM2S_IS_AXI 4ERR		MM2S_IS_DO NE
		15:8								
		23:16	MM2S_IS_CMDID[7:0]							
		31:24	MM2S_IS_CMDID[9:8]							
0x042C ... 0x04FF	Reserved									
0x0500	MM2S_AXI4BUSERR_CNTR	7:0	MM2S_AXI4BUSERR_CNTR[7:0]							
		15:8	MM2S_AXI4BUSERR_CNTR[15:8]							
		23:16								
		31:24								

8.1 MM2S_VERSN [\(Ask a Question\)](#)

Name: MM2S_VERSN
Offset: 0x400
Reset: 0x03000000
Property: Read-only

Version register. It will provide Major version and Minor version of the core.

Bit	31	30	29	28	27	26	25	24
	MAJOR[7:0]							
Access	R	R	R	R	R	R	R	R
Reset	0	0	0	0	0	0	1	1
Bit	23	22	21	20	19	18	17	16
	MINOR[7:0]							
Access	R	R	R	R	R	R	R	R
Reset	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8
Access								
Reset								
Bit	7	6	5	4	3	2	1	0
Access								
Reset								

Bits 31:24 – MAJOR[7:0] Major version of the core

Bits 23:16 – MINOR[7:0] Minor version of the core

8.2 MM2S_STS [\(Ask a Question\)](#)

Name: MM2S_STS
Offset: 0x414
Reset: 0x1
Property: Read-only

This register contains status of the MM2S data transfers.

Bit	31	30	29	28	27	26	25	24
							CMDID[9:8]	
Access							R	R
Reset							0	0

Bit	23	22	21	20	19	18	17	16
	CMDID[7:0]							
Access	R	R	R	R	R	R	R	R
Reset	0	0	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8
Access								
Reset								

Bit	7	6	5	4	3	2	1	0
					AXI4RRSP[1:0]			DONE
Access					R	R		R
Reset					0	0		1

Bits 25:16 – CMDID[9:0] Command ID. Provides command ID for which MM2S transfer is completed

Bits 3:2 – AXI4RRSP[1:0]

Value	Description
2b00	OK
2b01	EXOKAY
2b10	SLVERR
2b11	DECERR

Bit 0 – DONE

Value	Description
0b0	Indicates MM2S transfer is not completed
0b1	Indicates MM2S transfer is completed

8.3 MM2S_LEN [\(Ask a Question\)](#)

Name: MM2S_LEN
Offset: 0x418
Reset: 0x1
Property: Read/Write

This register contains MM2S length.

Bit	31	30	29	28	27	26	25	24
	MM2S_LEN[31:24]							
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0
Bit	23	22	21	20	19	18	17	16
	MM2S_LEN[23:16]							
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8
	MM2S_LEN[15:8]							
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0
Bit	7	6	5	4	3	2	1	0
	MM2S_LEN[7:0]							
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	1

Bits 31:0 – MM2S_LEN[31:0] Indicates the total number of bytes that are transmitted from the AXI4-Memory Mapped interface to the AXI4-Stream interface. Transfers can be from 1 to $(2^{32}) - 1$ bytes. You must not program 0 value to this register. When MM2S Command/Status FIFO Enable is not enabled, do not write in this register until you complete the current transfer. You can poll the MM2S Done bit of MM2S Status register to know the status of current transfer.

8.4 MM2S_INTRSRC [\(Ask a Question\)](#)

Name: MM2S_INTRSRC
Offset: 0x428
Reset: 0x0
Property: Read/Write

Interrupt source register

Bit	31	30	29	28	27	26	25	24
							MM2S_IS_CMDID[9:8]	
Access							R/W	R/W
Reset							0	0

Bit	23	22	21	20	19	18	17	16
	MM2S_IS_CMDID[7:0]							
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8
Access								
Reset								

Bit	7	6	5	4	3	2	1	0
						MM2S_IS_AXI4ERR		MM2S_IS_DONE
Access						R/W		R/W
Reset						0		0

Bits 25:16 – MM2S_IS_CMDID[9:0] Command ID. Provides command ID for which MM2S transfer is completed

Bit 2 – MM2S_IS_AXI4ERR

Value	Description
0b0	Indicates MM2S AXI4 Error is not detected. Writing 0 to this bit will have no effect
0b1	Indicates MM2S AXI4 Error is detected. Writing 1 to this bit will clear the bit

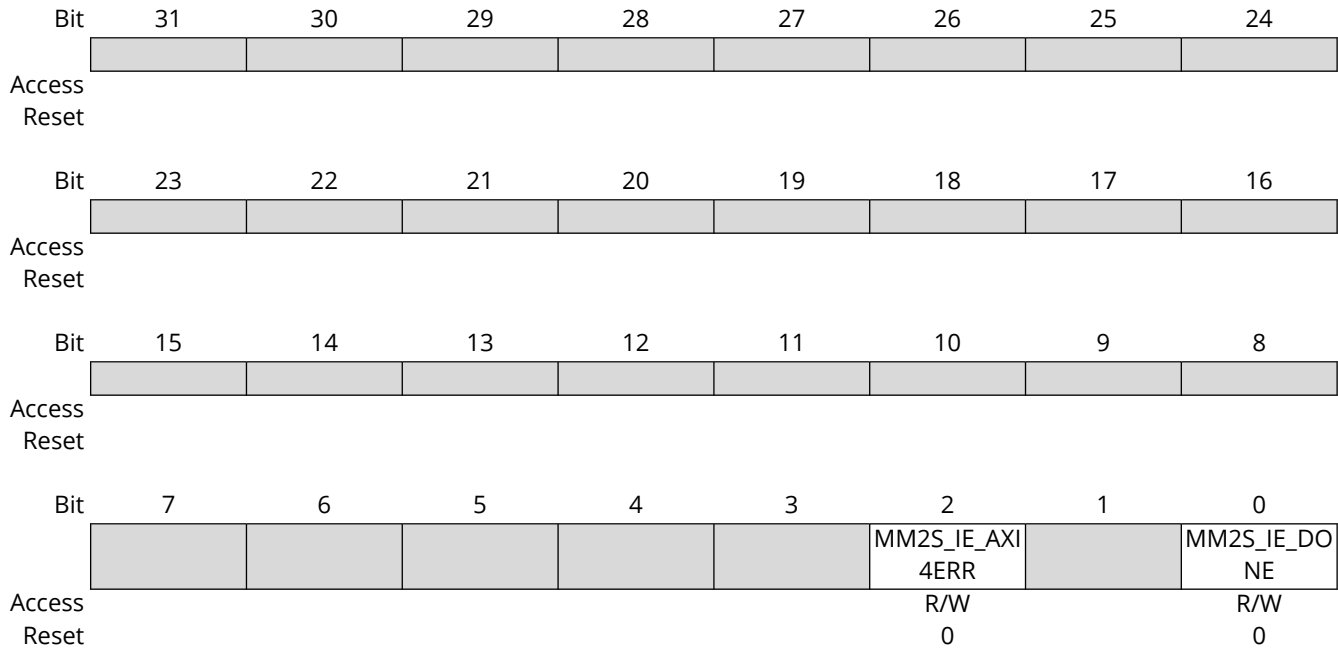
Bit 0 – MM2S_IS_DONE

Value	Description
0b0	Indicates MM2S transfer is not completed. Writing 0 to this bit will have no effect
0b1	Indicates MM2S transfer is completed. Writing 1 to this bit will clear the bit

8.5 MM2S_INTRENB [\(Ask a Question\)](#)

Name: MM2S_INTRENB
Offset: 0x424
Reset: 0x0
Property: Read/Write

Interrupt enable register



Bit 2 – MM2S_IE_AXI4ERR

Value	Description
0b0	Disable interrupt generation when AXI4 bus error is detected.
0b1	Enable interrupt generation when AXI4 bus error is detected

Bit 0 – MM2S_IE_DONE

Value	Description
0b0	Disable interrupt generation when MM2S done is asserted
0b1	Enable interrupt generation when MM2S done is asserted

8.6 MM2S_CTRL [\(Ask a Question\)](#)

Name: MM2S_CTRL
Offset: 0x410
Reset: 0x0
Property: Read/Write

MM2S control register. This register must be written after writing MM2S length, MM2S address0, and MM2S address1 (if GUI parameter MM2S Address Width is configured to more than 32) registers. When GUI parameter MM2S Command/Status FIFO Enable is not enabled, this register must not be written until the current transfer is completed. User can poll Done bit of MM2S status register to know the status of current transfer.

Bit	31	30	29	28	27	26	25	24
							CMDID[9:8]	
Access							R/W	R/W
Reset							0	0

Bit	23	22	21	20	19	18	17	16
	CMDID[7:0]							
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0

Bit	15	14	13	12	11	10	9	8
Access								
Reset								

Bit	7	6	5	4	3	2	1	0
						BURTP[1:0]		START
Access						R/W	R/W	R/W
Reset						0	0	0

Bits 25:16 – CMDID[9:0] User can assign an arbitrary value to this field. On completion of the MM2S transfer, IP core inserts this field into the MM2S status register. IP core do not support out of order operation. Command issued in the order must be reflected in the same order in the status

Bits 2:1 – BURTP[1:0] If the AXI4 length is “1”, FIXED and INCR bursts are equivalent. Fixed burst transfer should be used to access peripheral device which supports fixed address like FIFO. Increment burst transfer should be used to access peripheral device which supports incremental address like SRAM.

Value	Description
2b00	AXI4-Memory Mapped fixed burst
2b01	AXI4-Memory Mapped increment burst
2b10	Reserved
2b11	Reserved

Bit 0 – START When MM2S Command/Status FIFO Enable is enabled, CoreAXI4ProtoConv stores the descriptor into command FIFO. Descriptors consist of MM2S length, MM2S address0, MM2S address1, and MM2S Control register (bits 16 to 25) information. MM2S Start bit is a self-clearing bit.

Value	Description
0b0	There is no impact on MM2S data transfer
0b1	Initiates MM2S data transfer

8.7 MM2S_ADDR1 [\(Ask a Question\)](#)

Name: MM2S_ADDR1
Offset: 0x420
Reset: 0x0
Property: Read/Write

This register contains upper 32 bits of the start address of the MM2S data transfer

Bit	31	30	29	28	27	26	25	24
	MM2S_ADDR1[31:24]							
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0
Bit	23	22	21	20	19	18	17	16
	MM2S_ADDR1[23:16]							
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8
	MM2S_ADDR1[15:8]							
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0
Bit	7	6	5	4	3	2	1	0
	MM2S_ADDR1[7:0]							
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0

Bits 31:0 – MM2S_ADDR1[31:0] MM2S address 1. This register is used to configured upper 32 bits of start address for AXI4-Memory Mapped interface. This register must be programmed only when GUI parameter MM2S Address Width configured to more than 32

8.8 MM2S_AXI4BUSERR_CNTR [\(Ask a Question\)](#)

Name: MM2S_AXI4BUSERR_CNTR
Offset: 0x500
Reset: 0x0
Property: Read/Write

This register provides MM2S AXI4 bus error count.

Bit	31	30	29	28	27	26	25	24
Access								
Reset								
Bit	23	22	21	20	19	18	17	16
Access								
Reset								
Bit	15	14	13	12	11	10	9	8
	MM2S_AXI4BUSERR_CNTR[15:8]							
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0
Bit	7	6	5	4	3	2	1	0
	MM2S_AXI4BUSERR_CNTR[7:0]							
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0

Bits 15:0 – MM2S_AXI4BUSERR_CNTR[15:0] This register provides MM2S AXI4 bus error count. MM2S AXI4 bus error counter is incremented when MM2S AXI4WRSP (either SLVERR or DECERR) of MM2S status register is set to 1

8.9 MM2S_ADDR0 [\(Ask a Question\)](#)

Name: MM2S_ADDR0
Offset: 0x41C
Reset: 0x0
Property: Read/Write

This register contains lower 32 bits of the start address of the MM2S data transfer

Bit	31	30	29	28	27	26	25	24
	MM2S_ADDR0[31:24]							
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0
Bit	23	22	21	20	19	18	17	16
	MM2S_ADDR0[23:16]							
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8
	MM2S_ADDR0[15:8]							
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0
Bit	7	6	5	4	3	2	1	0
	MM2S_ADDR0[7:0]							
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0

Bits 31:0 – MM2S_ADDR0[31:0] MM2S address 0. This register is used to configured lower 32 bits address for AXI4-Memory Mapped interface. When GUI parameter MM2S Command/Status FIFO Enable is not enabled, this register must not be written until the current transfer is completed. User can poll Done bit of MM2S status register to know the status of current transfer MM2S address register0 and MM2S address register1 (only if GUI parameter MM2S Address Width is configured to more than 32) are used as start address to transfer total number of bytes configured in MM2S length register. If start address and total number of bytes crosses 4K address boundary or burst length is more than 256 then core breaks the MM2S transfer into multiple transfers internally. The CoreAXI4ProtoConv does not support unaligned transfers so start address must always be aligned with the MM2S Data Width.

9. Additional References [\(Ask a Question\)](#)

For updates and additional information about the software, devices, and hardware, visit the **Intellectual Property** pages on the [Microchip FPGAs and PLDs website](#).

9.1 Known Issues and Workarounds [\(Ask a Question\)](#)

There are no known limitations and workarounds for CoreAXI4ProtoConv v3.0.

9.2 Discontinued Features and Devices [\(Ask a Question\)](#)

There are no discontinued features and devices for CoreAXI4ProtoConv v3.0.

10. Device Utilization and Performance [\(Ask a Question\)](#)

This section provides the device utilization and performance of CoreAXI4ProtoConv's S2MM macro and CoreAXI4ProtoConv's MM2S macro.

10.1 CoreAXI4ProtoConv S2MM Device Utilization and Performance [\(Ask a Question\)](#)

The CoreAXI4ProtoConv's S2MM macro is implemented in the families listed in the following table.

The following table lists the CoreAXI4ProtoConv S2MM utilization when S2MM data width is 32 bits.

Table 10-1. CoreAXI4ProtoConv's S2MM Utilization (S2MM Data Width (in bits) = 32)

Device Details		Resources			Performance	RAMs	
Family	Device	4LUT	DFF	Logic Elements	ACLK	LSRAM	uSRAM
PolarFire [®] SoC	MPFS250T	1685	1003	1704	275	4	9
PolarFire	MPF300T	1470	1002	1537	265	4	9
RT PolarFire	RTPF500T	1470	1002	1537	265	4	9
RTG4 [™]	RT4G150	1603	1049	1649	125	4	6
SmartFusion [®] 2	M2S150TS	1636	1126	1690	170	4	6
IGLOO [®] 2	M2GL150TS	1636	1126	1690	170	4	6



Important: The data in the preceding table are achieved using Libero[®] SoC v2023.1.

1. The following top-level configuration GUI parameter have been modified from their default values: S2MM Data FIFO Depth = 2048.
2. Clock constraints used to achieve the performance numbers is: ACLK = 150 MHz.

The following table lists the CoreAXI4ProtoConv S2MM utilization when S2MM data width is 64 bits.

Table 10-2. CoreAXI4ProtoConv's S2MM Utilization (S2MM Data Width (in bits) = 64)

Device Details		Resources			Performance	RAMs	
Family	Device	4LUT	DFF	Logic Elements	ACLK	LSRAM	uSRAM
PolarFire [®] SoC	MPFS250T	1875	1197	1895	250	7	9
PolarFire	MPF300T	1694	1204	1769	260	7	9
RT PolarFire	RTPF500T	1694	1204	1769	260	7	9
RTG4 [™]	RT4G150	1863	1330	1913	130	8	6
SmartFusion [®] 2	M2S150TS	2132	1427	2175	170	8	6
IGLOO [®] 2	M2GL150TS	2132	1427	2175	170	8	6



Important: The data in the preceding table are achieved using Libero SoC v2023.1.

1. The following top-level configuration GUI parameter have been modified from their default values: S2MM Data FIFO Depth = 2048.
2. Clock constraints used to achieve the performance numbers is: ACLK = 150 MHz.

The following table lists the CoreAXI4ProtoConv S2MM utilization when S2MM data width is 128 bits.

Table 10-3. CoreAXI4ProtoConv's S2MM Utilization (S2MM Data Width (in bits) = 128)

Device Details		Resources			Performance	RAMs	
Family	Device	4LUT	DFF	Logic Elements	ACLK	LSRAM	uSRAM
PolarFire® SoC	MPFS250T	2153	1580	2216	240	13	8
PolarFire	MPF300T	2154	1580	2218	250	13	8
RT PolarFire	RTPF500T	2154	1580	2218	250	13	8
RTG4™	RT4G150	2400	1717	2473	130	15	5
SmartFusion® 2	M2S150TS	2761	1873	2811	140	15	5
IGLOO® 2	M2GL150TS	2761	1873	2811	140	15	5



Important: The data in the preceding table are achieved using Libero SoC v2023.1.

1. The following top-level configuration GUI parameter have been modified from their default values: S2MM Data FIFO Depth = 2048.
2. Clock constraints used to achieve the performance numbers is: ACLK = 150 MHz.

The following table lists the CoreAXI4ProtoConv S2MM utilization when S2MM data width is 256 bits.

Table 10-4. CoreAXI4ProtoConv's S2MM Utilization (S2MM Data Width (in bits) = 256)

Device Details		Resources			Performance	RAMs	
Family	Device	4LUT	DFF	Logic Elements	ACLK	LSRAM	uSRAM
PolarFire® SoC	MPFS250T	3207	2433	3279	220	26	8
PolarFire	MPF300T	3181	2433	3263	220	26	8
RT PolarFire	RTPF500T	3181	2433	3263	220	26	8
RTG4®	RT4G150	3642	2604	3678	125	29	5
SmartFusion® 2	M2S150TS	3732	2898	3842	145	29	5
IGLOO® 2	M2GL150TS	3732	2898	3842	145	29	5



Important: The data in the preceding table are achieved using Libero SoC v2023.1.

1. The following top-level configuration GUI parameter have been modified from their default values: S2MM Data FIFO Depth = 2048.
2. Clock constraints used to achieve the performance numbers is: ACLK = 150 MHz.

The following table lists the CoreAXI4ProtoConv S2MM utilization when S2MM data width is 512 bits.

Table 10-5. CoreAXI4ProtoConv's S2MM Utilization (S2MM Data Width (in bits) = 512)

Device Details		Resources			Performance	RAMs	
Family	Device	4LUT	DFF	Logic Elements	ACLK	LSRAM	uSRAM
PolarFire® SoC	MPFS250T	5290	4137	5390	220	52	8
PolarFire	MPF300T	5275	4137	5374	220	52	8

.....continued

Device Details		Resources			Performance	RAMs	
Family	Device	4LUT	DFF	Logic Elements	ACLK	LSRAM	uSRAM
RT PolarFire	RTPF500T	5275	4137	5374	220	52	8
RTG4™	RT4G150	5580	4412	5690	115	57	5
SmartFusion® 2	M2S150TS	6102	4998	6269	125	57	5
IGLOO® 2	M2GL150TS	6102	4998	6269	125	57	5



Important: The data in the preceding table are achieved using Libero SoC v2023.1.

1. The following top-level configuration GUI parameter have been modified from their default values: S2MM Data FIFO Depth = 2048.
2. Clock constraints used to achieve the performance numbers is: ACLK = 150 MHz.

The following table lists the CoreAXI4ProtoConv S2MM utilization when S2MM data width is 32 bits and burst length enable is 1.

Table 10-6. CoreAXI4ProtoConv's S2MM Utilization (S2MM Data Width (in bits) = 32 and S2MM Undefined Burst Length Enable = 1)

Device Details		Resources			Performance	RAMs	
Family	Device	4LUT	DFF	Logic Elements	ACLK	LSRAM	uSRAM
PolarFire® SoC	MPFS250T	1430	1085	1520	260	4	6
PolarFire	MPF300T	1433	1085	1514	265	4	8
RT PolarFire	RTPF500T	1433	1085	1514	265	4	8
RTG4™	RT4G150	1751	1242	1817	135	5	6
SmartFusion® 2	M2S150TS	1815	1290	1902	170	5	6
IGLOO® 2	M2GL150TS	1815	1290	1902	170	5	6



Important: The data in the preceding table is achieved using default synthesis and layout settings.

1. The following top-level configuration GUI parameters have been modified from their default values:
 - S2MM Data FIFO Depth = 2048
 - S2MM Undefined Burst Length Enable = 1
 - S2MM Burst Length = 64
2. Clock constraints used to achieve the performance numbers is: ACLK = 150 MHz

The following table lists the CoreAXI4ProtoConv S2MM utilization when S2MM data width is 64 bits and burst length enable is 1.

Table 10-7. CoreAXI4ProtoConv's S2MM Utilization (S2MM Data Width (in bits) = 64 and S2MM Undefined Burst Length Enable = 1)

Device Details		Resources			Performance	RAMs	
Family	Device	4LUT	DFF	Logic Elements	ACLK	LSRAM	uSRAM
PolarFire® SoC	MPFS250T	1830	1335	1930	240	8	8
PolarFire	MPF300T	1829	1335	1924	240	8	8
RT PolarFire	RTPF500T	1829	1335	1924	240	8	8
RTG4™	RT4G150	1903	1492	1993	125	9	6
SmartFusion® 2	M2S150TS	2026	1574	2138	165	9	6
IGLOO® 2	M2GL150TS	2026	1574	2138	165	9	6



Important: The data in the preceding table is achieved using default synthesis and layout settings.

- The following top-level configuration GUI parameters have been modified from their default values:
 - S2MM Data FIFO Depth = 2048
 - S2MM Undefined Burst Length Enable = 1
 - S2MM Burst Length = 64
- Clock constraints used to achieve the performance numbers is: ACLK = 150 MHz

The following table lists the CoreAXI4ProtoConv S2MM utilization when S2MM data width is 128 bits and burst length enable is 1.

Table 10-8. CoreAXI4ProtoConv's S2MM Utilization (S2MM Data Width (in bits) = 128 and S2MM Undefined Burst Length Enable = 1)

Device Details		Resources			Performance	RAMs	
Family	Device	4LUT	DFF	Logic Elements	ACLK	LSRAM	uSRAM
PolarFire® SoC	MPFS250T	2367	1813	2478	240	15	8
PolarFire	MPF300T	2367	1813	2478	240	15	8
RT PolarFire	RTPF500T	2367	1813	2478	240	15	8
RTG4™	RT4G150	2572	2000	2700	125	17	6
SmartFusion® 2	M2S150TS	2760	2169	2881	160	17	6
IGLOO® 2	M2GL150TS	2760	2169	2881	160	17	6

 **Important:** The data in the preceding table is achieved using default synthesis and layout settings.

1. The following top-level configuration GUI parameters have been modified from their default values:
 - S2MM Data FIFO Depth = 2048
 - S2MM Undefined Burst Length Enable = 1
 - S2MM Data Width = 128
 - S2MM Burst Length = 64
2. Clock constraints used to achieve the performance numbers is: ACLK = 150 MHz

The following table lists the CoreAXI4ProtoConv S2MM utilization when S2MM data width is 256 bits and burst length enable is 1.

Table 10-9. CoreAXI4ProtoConv's S2MM Utilization (S2MM Data Width (in bits) = 256 and S2MM Undefined Burst Length Enable = 1)

Device Details		Resources			Performance	RAMs	
Family	Device	4LUT	DFF	Logic Elements	ACLK	LSRAM	uSRAM
PolarFire [®] SoC	MPFS250T	3656	2748	3755	220	29	8
PolarFire	MPF300T	3650	2750	3764	210	29	8
RT PolarFire	RTPF500T	3650	2750	3764	210	29	8
RTG4 [™]	RT4G150	3959	3006	4066	120	33	6
SmartFusion [®] 2	M2S150TS	4244	3330	4364	120	33	6
IGLOO [®] 2	M2GL150TS	4244	3330	4364	120	33	6

 **Important:** The data in the preceding table is achieved using default synthesis and layout settings.

1. The following top-level configuration GUI parameters have been modified from their default values:
 - S2MM Data FIFO Depth = 2048
 - S2MM Undefined Burst Length Enable = 1
 - S2MM Data Width = 128
 - S2MM Burst Length = 64
2. Clock constraints used to achieve the performance numbers is: ACLK = 150 MHz

The following table lists the CoreAXI4ProtoConv S2MM utilization when S2MM data width is 512 bits and burst length enable is 1.

Table 10-10. CoreAXI4ProtoConv's S2MM Utilization (S2MM Data Width (in bits) = 512 and S2MM Undefined Burst Length Enable = 1)

Device Details		Resources			Performance	RAMs	
Family	Device	4LUT	DFF	Logic Elements	ACLK	LSRAM	uSRAM
PolarFire [®] SoC	MPFS250T	6262	4673	6397	200	58	8

.....continued

Device Details		Resources			Performance	RAMs	
Family	Device	4LUT	DFF	Logic Elements	ACLK	LSRAM	uSRAM
PolarFire	MPF300T	6262	4673	6397	195	58	8
RT PolarFire	RTPF500T	6262	4673	6397	195	58	8
RTG4™	RT4G150	6608	5039	6737	115	65	6
SmartFusion® 2	M2S150TS	7169	5664	7363	100	65	6
IGLOO® 2	M2GL150TS	7169	5664	7363	100	65	6



Important: The data in the preceding table is achieved using default synthesis and layout settings.

- The following top-level configuration GUI parameters have been modified from their default values:
 - S2MM Data FIFO Depth = 2048
 - S2MM Undefined Burst Length Enable = 1
 - S2MM Data Width = 512
 - S2MM Burst Length = 64
- Clock constraints used to achieve the performance numbers is: ACLK = 150 MHz

The following table lists the CoreAXI4ProtoConv S2MM utilization when S2MM data width is 32 bits, S2MM command/status FIFO enable is 0, and S2MM data FIFO enable is 0.

Table 10-11. CoreAXI4ProtoConv's S2MM Utilization (S2MM Data Width (in bits) = 32, S2MM Command/Status FIFO Enable = 0 and S2MM Data FIFO Enable = 0)

Device Details		Resources			Performance	RAMs	
Family	Device	4LUT	DFF	Logic Elements	ACLK	LSRAM	uSRAM
PolarFire® SoC	MPFS250T	751	385	767	300	0	0
PolarFire	MPF300T	751	385	765	300	0	0
RT PolarFire	RTPF500T	751	385	765	300	0	0
RTG4™	RT4G150	752	385	771	170	0	0
SmartFusion® 2	M2S150TS	744	385	763	175	0	0
IGLOO® 2	M2GL150TS	744	385	763	175	0	0



Important: The data in the preceding table is achieved using default synthesis and layout settings.

- The following top-level configuration GUI parameters have been modified from their default values:
 - S2MM Data FIFO Enable = 0
 - S2MM Command/Status FIFO Enable = 0
- Clock constraints used to achieve the performance numbers is: ACLK = 150 MHz

The following table lists the CoreAXI4ProtoConv S2MM utilization when S2MM data width is 64 bits, S2MM command/status FIFO enable is 0, and S2MM data FIFO enable is 0.

Table 10-12. CoreAXI4ProtoConv's S2MM Utilization (S2MM Data Width (in bits) = 64, S2MM Command/Status FIFO Enable = 0 and S2MM Data FIFO Enable = 0)

Device Details		Resources			Performance	RAMs	
Family	Device	4LUT	DFF	Logic Elements	ACLK	LSRAM	uSRAM
PolarFire [®] SoC	MPFS250T	733	384	752	330	0	0
PolarFire	MPF300T	733	384	752	330	0	0
RT PolarFire	RTPF500T	733	384	752	330	0	0
RTG4 [™]	RT4G150	736	388	747	180	0	0
SmartFusion [®] 2	M2S150TS	748	400	761	195	0	0
IGLOO [®] 2	M2GL150TS	748	400	761	195	0	0



Important: The data in the preceding table is achieved using default synthesis and layout settings.

- The following top-level configuration GUI parameters have been modified from their default values:
 - S2MM Data FIFO Enable = 0
 - S2MM Command/Status FIFO Enable = 0
 - S2MM Data Width (in bits) = 64
- Clock constraints used to achieve the performance numbers is: ACLK = 150 MHz

The following table lists the CoreAXI4ProtoConv S2MM utilization when S2MM data width is 128 bits, S2MM command/status FIFO enable is 0, and S2MM data FIFO enable is 0.

Table 10-13. CoreAXI4ProtoConv's S2MM Utilization (S2MM Data Width (in bits) = 128, S2MM Command/Status FIFO Enable = 0 and S2MM Data FIFO Enable = 0)

Device Details		Resources			Performance	RAMs	
Family	Device	4LUT	DFF	Logic Elements	ACLK	LSRAM	uSRAM
PolarFire [®] SoC	MPFS250T	717	383	737	380	0	0
PolarFire	MPF300T	717	383	737	380	0	0
RT PolarFire	RTPF500T	717	383	737	380	0	0
RTG4 [™]	RT4G150	784	383	794	200	0	0
SmartFusion [®] 2	M2S150TS	774	394	779	180	0	0
IGLOO [®] 2	M2GL150TS	774	394	779	180	0	0

➔ Important: The data in the preceding table is achieved using default synthesis and layout settings.

- The following top-level configuration GUI parameters have been modified from their default values:
 - S2MM Data FIFO Enable = 0
 - S2MM Command/Status FIFO Enable = 0
 - S2MM Data Width (in bits) = 128
- Clock constraints used to achieve the performance numbers is: ACLK = 150 MHz

The following table lists the CoreAXI4ProtoConv S2MM utilization when S2MM data width is 256 bits, S2MM command/status FIFO enable is 0, and S2MM data FIFO enable is 0.

Table 10-14. CoreAXI4ProtoConv's S2MM Utilization (S2MM Data Width (in bits) = 256, S2MM Command/Status FIFO Enable = 0 and S2MM Data FIFO Enable = 0)

Device Details		Resources			Performance	RAMs	
Family	Device	4LUT	DFF	Logic Elements	ACLK	LSRAM	uSRAM
PolarFire [®] SoC	MPFS250T	799	382	821	300	0	0
PolarFire	MPF300T	799	382	821	300	0	0
RT PolarFire	RTPF500T	799	382	821	300	0	0
RTG4 [™]	RT4G150	830	382	839	190	0	0
SmartFusion [®] 2	M2S150TS	828	388	846	190	0	0
IGLOO [®] 2	M2GL150TS	828	388	846	190	0	0

➔ Important: The data in the preceding table is achieved using default synthesis and layout settings.

- The following top-level configuration GUI parameters have been modified from their default values:
 - S2MM Data FIFO Enable = 0
 - S2MM Command/Status FIFO Enable = 0
 - S2MM Data Width (in bits) = 256
- Clock constraints used to achieve the performance numbers is: ACLK = 150 MHz

The following table lists the CoreAXI4ProtoConv S2MM utilization when S2MM data width is 512bits, S2MM command/status FIFO enable is 0, and S2MM data FIFO enable is 0.

Table 10-15. CoreAXI4ProtoConv's S2MM Utilization (S2MM Data Width (in bits) = 512, S2MM Command/Status FIFO Enable = 0 and S2MM Data FIFO Enable = 0)

Device Details		Resources			Performance	RAMs	
Family	Device	4LUT	DFF	Logic Elements	ACLK	LSRAM	uSRAM
PolarFire [®] SoC	MPFS250T	922	381	939	330	0	0
PolarFire	MPF300T	922	381	939	330	0	0
RT PolarFire	RTPF500T	922	381	939	330	0	0
RTG4 [™]	RT4G150	939	381	949	200	0	0

.....continued

Device Details		Resources			Performance	RAMs	
Family	Device	4LUT	DFF	Logic Elements	ACLK	LSRAM	uSRAM
SmartFusion® 2	M2S150TS	941	386	955	175	0	0
IGLOO® 2	M2GL150TS	941	386	955	175	0	0

 **Important:** The data in the preceding table is achieved using default synthesis and layout settings.

- The following top-level configuration GUI parameters have been modified from their default values:
 - S2MM Data FIFO Enable = 0
 - S2MM Command/Status FIFO Enable = 0
 - S2MM Data Width (in bits) = 512
- Clock constraints used to achieve the performance numbers is: ACLK = 150 MHz

The CoreAXI4ProtoConv's S2MM macro is implemented in the families listed in the following table. S2MM Packet drop due to error = 1, and S2MM store and forward mode is enabled.

Table 10-16. CoreAXI4ProtoConv's S2MM Utilization (S2MM Data Width (in bits) = 32 S2MM Packet drop due to error = 1, and S2MM store and forward mode is enabled)

Device Details		Resources			Performance	RAMs	
Family	Device	4LUT	DFF	Logic Elements	ACLK	LSRAM	uSRAM
PolarFire® SoC	MPFS250T	1664	1060	1731	260	4	9
PolarFire	MPF300T	1664	1060	1711	250	4	9
RT PolarFire	RTPF500T	1664	1060	1711	250	4	9
RTG4™	RT4G150	1809	1154	1847	125	4	6
SmartFusion® 2	M2S150TS	1849	1182	1915	155	4	6
IGLOO® 2	M2GL150TS	1849	1182	1915	155	4	6

 **Important:** The data in the preceding table are achieved using Libero SoC v2023.1.

- The following top-level configuration GUI parameters have been modified from their default values:
 - S2MM Packet drop due to error = 1
 - S2MM store and forward mode = 1
 - S2MM Data FIFO Depth = 2048
- Clock constraints used to achieve the performance numbers is: ACLK = 150 MHz.

The CoreAXI4ProtoConv's S2MM macro is implemented in the families listed in the following table. S2MM Packet drop due to error = 1, and S2MM store and forward mode is enabled.

Table 10-17. CoreAXI4ProtoConv's S2MM Utilization (S2MM Data Width (in bits) = 64 S2MM Packet drop due to error = 1, and S2MM store and forward mode is enabled)

Device Details		Resources			Performance	RAMs	
Family	Device	4LUT	DFF	Logic Elements	ACLK	LSRAM	uSRAM
PolarFire [®] SoC	MPFS250T	1875	1262	1954	235	7	9
PolarFire	MPF300T	1875	1262	1934	230	7	9
RT PolarFire	RTPF500T	1875	1262	1934	230	7	9
RTG4 [™]	RT4G150	2066	1399	2131	125	8	6
SmartFusion [®] 2	M2S150TS	2234	1470	2278	162	8	6
IGLOO [®] 2	M2GL150TS	2234	1470	2278	162	8	6



Important: The data in the preceding table are achieved using Libero SoC v2023.1.

- The following top-level configuration GUI parameters have been modified from their default values:
 - S2MM Packet drop due to error = 1
 - S2MM store and forward mode = 1
 - S2MM Data Width (in bits) = 64
 - S2MM Data FIFO Depth = 2048
- Clock constraints used to achieve the performance numbers is: ACLK = 150 MHz.

The CoreAXI4ProtoConv's S2MM macro is implemented in the families listed in the following table. S2MM Packet drop due to error = 1, and S2MM store and forward mode is enabled.

Table 10-18. CoreAXI4ProtoConv's S2MM Utilization (S2MM Data Width (in bits) = 128 S2MM Packet drop due to error = 1, and S2MM store and forward mode is enabled)

Device Details		Resources			Performance	RAMs	
Family	Device	4LUT	DFF	Logic Elements	ACLK	LSRAM	uSRAM
PolarFire [®] SoC	MPFS250T	2346	1639	2455	210	13	8
PolarFire	MPF300T	2346	1639	2435	210	13	8
RT PolarFire	RTPF500T	2346	1639	2435	210	13	8
RTG4 [™]	RT4G150	2673	1775	2728	120	15	5
SmartFusion [®] 2	M2S150TS	2748	1929	2820	155	15	5
IGLOO [®] 2	M2GL150TS	2748	1929	2820	155	15	5

 **Important:** The data in the preceding table are achieved using Libero SoC v2023.1.

- The following top-level configuration GUI parameters have been modified from their default values:
 - S2MM Data FIFO Depth = 2048
 - S2MM Data Width (in bits) = 128
 - S2MM store and forward mode = 1
 - S2MM Packet drop due to error = 1
- Clock constraints used to achieve the performance numbers is: ACLK = 150 MHz.

The CoreAXI4ProtoConv's S2MM macro is implemented in the families listed in the following table. S2MM Packet drop due to error = 1, and S2MM store and forward mode is enabled.

Table 10-19. CoreAXI4ProtoConv's S2MM Utilization (S2MM Data Width (in bits) = 256 S2MM Packet drop due to error = 1, and S2MM store and forward mode is enabled)

Device Details		Resources			Performance	RAMs	
Family	Device	4LUT	DFF	Logic Elements	ACLK	LSRAM	uSRAM
PolarFire® SoC	MPFS250T	3377	2491	3455	220	26	8
PolarFire	MPF300T	3377	2491	3455	220	26	8
RT PolarFire	RTPF500T	3377	2491	3455	220	26	8
RTG4™	RT4G150	3697	2665	3777	120	29	5
SmartFusion® 2	M2S150TS	4056	2946	4137	140	29	5
IGLOO® 2	M2GL150TS	4056	2946	4137	140	29	5

 **Important:** The data in the preceding table are achieved using Libero SoC v2023.1.

- The following top-level configuration GUI parameters have been modified from their default values:
 - S2MM Packet drop due to error = 1
 - S2MM store and forward mode = 1
 - S2MM Data Width (in bits) = 256
 - S2MM Data FIFO Depth = 2048
- Clock constraints used to achieve the performance numbers is: ACLK = 150 MHz.

The CoreAXI4ProtoConv's S2MM macro is implemented in the families listed in the following table. S2MM Packet drop due to error = 1, and S2MM store and forward mode is enabled.

Table 10-20. CoreAXI4ProtoConv's S2MM Utilization (S2MM Data Width (in bits) = 512 S2MM Packet drop due to error = 0, and S2MM store and forward mode is enabled)

Device Details		Resources			Performance	RAMs	
Family	Device	4LUT	DFF	Logic Elements	ACLK	LSRAM	uSRAM
PolarFire® SoC	MPFS250T	5461	4195	5555	200	52	8

.....continued

Device Details		Resources			Performance	RAMs	
Family	Device	4LUT	DFF	Logic Elements	ACLK	LSRAM	uSRAM
PolarFire	MPF300T	5461	4195	5567	200	52	8
RT PolarFire	RTPF500T	5461	4195	5567	200	52	8
RTG4™	RT4G150	5773	4449	5878	115	57	5
SmartFusion® 2	M2S150TS	6437	5017	6559	130	57	5
IGLOO® 2	M2GL150TS	6437	5017	6559	130	57	5



Important: The data in the preceding table are achieved using Libero SoC v2023.1.

- The following top-level configuration GUI parameters have been modified from their default values:
 - S2MM Packet drop due to error = 1
 - S2MM store and forward mode = 1
 - S2MM Data Width (in bits) = 512
 - S2MM Data FIFO Depth = 2048
- Clock constraints used to achieve the performance numbers is: ACLK = 150 MHz.

The CoreAXI4ProtoConv's S2MM macro is implemented in the families listed in the following table. S2MM Packet drop due to overflow = 1, and S2MM store and forward mode is enabled.

Table 10-21. CoreAXI4ProtoConv's S2MM Utilization (S2MM Data Width (in bits) = 32 S2MM Packet drop due to overflow = 1, and S2MM store and forward mode is enabled)

Device Details		Resources			Performance	RAMs	
Family	Device	4LUT	DFF	Logic Elements	ACLK	LSRAM	uSRAM
PolarFire® SoC	MPFS250T	1653	1074	1703	260	4	9
PolarFire	MPF300T	1653	1074	1703	260	4	9
RT PolarFire	RTPF500T	1653	1074	1703	260	4	9
RTG4™	RT4G150	1925	1162	1959	130	4	6
SmartFusion® 2	M2S150TS	1832	1205	1895	170	4	6
IGLOO® 2	M2GL150TS	1832	1205	1895	170	4	6



Important: The data in the preceding table are achieved using Libero SoC v2023.1.

- The following top-level configuration GUI parameters have been modified from their default values:
 - S2MM Packet drop due to overflow = 1
 - S2MM store and forward mode = 1
 - S2MM Data FIFO Depth = 2048
- Clock constraints used to achieve the performance numbers is: ACLK = 150 MHz.

The CoreAXI4ProtoConv's S2MM macro is implemented in the families listed in the following table. S2MM Packet drop due to overflow = 1, and S2MM store and forward mode is enabled.

Table 10-22. CoreAXI4ProtoConv's S2MM Utilization (S2MM Data Width (in bits) = 64 S2MM Packet drop due to overflow = 1, and S2MM store and forward mode is enabled)

Device Details		Resources			Performance	RAMs	
Family	Device	4LUT	DFF	Logic Elements	ACLK	LSRAM	uSRAM
PolarFire [®] SoC	MPFS250T	1903	1275	1974	240	7	9
PolarFire	MPF300T	1903	1275	1974	240	7	9
RT PolarFire	RTPF500T	1903	1275	1974	240	7	9
RTG4 [™]	RT4G150	2044	1403	2092	120	8	6
SmartFusion [®] 2	M2S150TS	2149	1484	2208	160	8	6
IGLOO [®] 2	M2GL150TS	2149	1484	2208	160	8	6



Important: The data in the tables are achieved using Libero SoC v2023.1.

- The following top-level configuration GUI parameters have been modified from their default values:
 - S2MM Packet drop due to overflow = 1
 - S2MM Data FIFO Depth = 2048
 - S2MM Data Width (in bits) = 64
 - S2MM store and forward mode = 1
- Clock constraints used to achieve the performance numbers is: ACLK = 150 MHz.

The CoreAXI4ProtoConv's S2MM macro is implemented in the families listed in the following table. S2MM Packet drop due to overflow = 1, and S2MM store and forward mode is enabled.

Table 10-23. CoreAXI4ProtoConv's S2MM Utilization (S2MM Data Width (in bits) = 128 S2MM Packet drop due to overflow = 1, and S2MM store and forward mode is enabled)

Device Details		Resources			Performance	RAMs	
Family	Device	4LUT	DFF	Logic Elements	ACLK	LSRAM	uSRAM
PolarFire [®] SoC	MPFS250T	2348	1643	2409	220	13	8
PolarFire	MPF300T	2348	1643	2409	220	13	8
RT PolarFire	RTPF500T	2348	1643	2409	220	13	8
RTG4 [™]	RT4G150	2673	1775	2728	120	15	5
SmartFusion [®] 2	M2S150TS	2640	1920	2720	145	15	5
IGLOO [®] 2	M2GL150TS	2640	1920	2720	145	15	5



Important: The data in the preceding table are achieved using Libero SoC v2023.1.

- The following top-level configuration GUI parameters have been modified from their default values:
 - S2MM Data FIFO Depth = 2048
 - S2MM Data Width (in bits) = 128
 - S2MM store and forward mode = 1
 - S2MM Packet drop due to overflow = 1
- Clock constraints used to achieve the performance numbers is: ACLK = 150 MHz.

The CoreAXI4ProtoConv's S2MM macro is implemented in the families listed in the following table. S2MM Packet drop due to overflow = 1, and S2MM store and forward mode is enabled.

Table 10-24. CoreAXI4ProtoConv's S2MM Utilization (S2MM Data Width (in bits) = 256 S2MM Packet drop due to overflow = 1, and S2MM store and forward mode is enabled)

Device Details		Resources			Performance	RAMs	
Family	Device	4LUT	DFF	Logic Elements	ACLK	LSRAM	uSRAM
PolarFire [®] SoC	MPFS250T	3385	2496	3465	210	26	8
PolarFire	MPF300T	3385	2496	3465	210	26	8
RT PolarFire	RTPF500T	3385	2496	3465	210	26	8
RTG4 [™]	RT4G150	3697	2665	3777	120	29	5
SmartFusion [®] 2	M2S150TS	3865	2953	3975	150	29	5
IGLOO [®] 2	M2GL150TS	3865	2953	3975	150	29	5



Important: The data in the preceding table are achieved using Libero SoC v2023.1.

- The following top-level configuration GUI parameters have been modified from their default values:
 - S2MM Packet drop due to overflow = 1
 - S2MM store and forward mode = 1
 - S2MM Data Width (in bits) = 256
 - S2MM Data FIFO Depth = 2048
- Clock constraints used to achieve the performance numbers is: ACLK = 150 MHz.

The CoreAXI4ProtoConv's S2MM macro is implemented in the families listed in the following table. S2MM Packet drop due to overflow = 1, and S2MM store and forward mode is enabled.

Table 10-25. CoreAXI4ProtoConv's S2MM Utilization (S2MM Data Width (in bits) = 512 S2MM Packet drop due to overflow = 1, and S2MM store and forward mode is enabled)

Device Details		Resources			Performance	RAMs	
Family	Device	4LUT	DFF	Logic Elements	ACLK	LSRAM	uSRAM
PolarFire [®] SoC	MPFS250T	5594	4999	5649	200	52	8

.....continued

Device Details		Resources			Performance	RAMs	
Family	Device	4LUT	DFF	Logic Elements	ACLK	LSRAM	uSRAM
PolarFire	MPF300T	5594	4999	5649	200	52	8
RT PolarFire	RTPF500T	5594	4999	5649	200	52	8
RTG4™	RT4G150	5773	4449	5878	115	57	5
SmartFusion® 2	M2S150TS	6311	5020	6449	130	57	5
IGLOO® 2	M2GL150TS	6311	5020	6449	130	57	5



Important: The data in the preceding table are achieved using Libero SoC v2023.1.

- The following top-level configuration GUI parameters have been modified from their default values:
 - S2MM Packet drop due to overflow = 1
 - S2MM store and forward mode = 1
 - S2MM Data Width (in bits) = 512
 - S2MM Data FIFO Depth = 2048
- Clock constraints used to achieve the performance numbers is: ACLK = 150 MHz.

10.2 CoreAXI4ProtoConv MM2S Device and Utilization Performance [\(Ask a Question\)](#)

The CoreAXI4ProtoConv's MM2S macro is implemented in the families listed in the following tables. The following table lists the CoreAXI4ProtoConv MM2S utilization when MM2S data width is 32 bits.

Table 10-26. CoreAXI4ProtoConv's MM2S Utilization (MM2S Data Width (in bits) = 32)

Device Details		Resources			Performance	RAMs	
Family	Device	4LUT	DFF	Logic Elements	ACLK	LSRAM	uSRAM
PolarFire® SoC	MPFS250T	1678	1000	1697	225	4	9
PolarFire	MPF300T	1678	1000	1697	265	4	9
RT PolarFire	RTPF500T	1678	1000	1697	265	4	9
RTG4™	RT4G150	1619	1086	1668	125	4	6
SmartFusion® 2	M2S150TS	1627	1124	1686	165	4	6
IGLOO® 2	M2GL150TS	1627	1124	1686	165	4	6



Important: The data in all the preceding tables are achieved using Libero® SoC v2023.1.

- The following top-level configuration GUI parameter have been modified from their default values: MM2S Data FIFO Depth = 2048.
- Clock constraints used to achieve the performance numbers is: ACLK = 150 MHz.

The following table lists the CoreAXI4ProtoConv MM2S utilization when MM2S data width is 64 bits.

Table 10-27. CoreAXI4ProtoConv's MM2S Utilization (MM2S Data Width (in bits) = 64)

Device Details		Resources			Performance	RAMs	
Family	Device	4LUT	DFF	Logic Elements	ACLK	LSRAM	uSRAM
PolarFire [®] SoC	MPFS250T	1699	1204	1751	230	7	9
PolarFire	MPF300T	1699	1204	1751	230	7	9
RT PolarFire	RTPF500T	1699	1204	1751	230	7	9
RTG4 [™]	RT4G150	2024	1323	2025	120	8	6
SmartFusion [®] 2	M2S150TS	1966	1406	2016	165	8	6
IGLOO [®] 2	M2GL150TS	1966	1406	2016	165	8	6



Important: The data in all the preceding tables are achieved using Libero SoC v2023.1.

1. The following top-level configuration GUI parameter have been modified from their default values: MM2S Data FIFO Depth = 2048.
2. Clock constraints used to achieve the performance numbers is: ACLK = 150 MHz.

The following table lists the CoreAXI4ProtoConv MM2S utilization when MM2S data width is 128 bits.

Table 10-28. CoreAXI4ProtoConv's MM2S Utilization (MM2S Data Width (in bits) = 128)

Device Details		Resources			Performance	RAMs	
Family	Device	4LUT	DFF	Logic Elements	ACLK	LSRAM	uSRAM
PolarFire [®] SoC	MPFS250T	2130	1565	2198	210	13	8
PolarFire	MPF300T	2130	1565	2198	210	13	8
RT PolarFire	RTPF500T	2130	1565	2198	210	13	8
RTG4 [™]	RT4G150	2461	1697	2482	120	15	5
SmartFusion [®] 2	M2S150TS	2643	1841	2703	154	15	5
IGLOO [®] 2	M2GL150TS	2643	1841	2703	154	15	5



Important: The data in all the preceding tables are achieved using Libero SoC v2023.1.

1. The following top-level configuration GUI parameter have been modified from their default values: MM2S Data FIFO Depth = 2048.
2. Clock constraints used to achieve the performance numbers is: ACLK = 150 MHz.

The following table lists the CoreAXI4ProtoConv MM2S utilization when MM2S data width is 256 bits.

Table 10-29. CoreAXI4ProtoConv's MM2S Utilization (MM2S Data Width (in bits) = 256)

Device Details		Resources			Performance	RAMs	
Family	Device	4LUT	DFF	Logic Elements	ACLK	LSRAM	uSRAM
PolarFire [®] SoC	MPFS250T	3117	2425	3191	210	26	8
PolarFire	MPF300T	3117	2425	3191	220	26	8
RT PolarFire	RTPF500T	3117	2425	3191	220	26	8

.....continued

Device Details		Resources			Performance	RAMs	
Family	Device	4LUT	DFF	Logic Elements	ACLK	LSRAM	uSRAM
RTG4™	RT4G150	3438	2593	3481	120	29	5
SmartFusion® 2	M2S150TS	3638	2891	3722	130	29	5
IGLOO® 2	M2GL150TS	3638	2891	3722	130	29	5



Important: The data in all the preceding tables are achieved using Libero SoC v2023.1.

1. The following top-level configuration GUI parameter have been modified from their default values: MM2S Data FIFO Depth = 2048.
2. Clock constraints used to achieve the performance numbers is: ACLK = 150 MHz.

The following table lists the CoreAXI4ProtoConv MM2S utilization when MM2S data width is 512 bits.

Table 10-30. CoreAXI4ProtoConv's MM2S Utilization (MM2S Data Width (in bits) = 512)

Device Details		Resources			Performance	RAMs	
Family	Device	4LUT	DFF	Logic Elements	ACLK	LSRAM	uSRAM
PolarFire® SoC	MPFS250T	5150	4135	5237	200	52	8
PolarFire	MPF300T	5150	4135	5237	200	52	8
RT PolarFire	RTPF500T	5150	4135	5237	200	52	8
RTG4™	RT4G150	5449	4382	5548	115	57	5
SmartFusion® 2	M2S150TS	5938	4960	6105	120	57	5
IGLOO® 2	M2GL150TS	5938	4960	6105	120	57	5



Important: The data in all the preceding tables are achieved using Libero SoC v2023.1.

1. The following top-level configuration GUI parameter have been modified from their default values: MM2S Data FIFO Depth = 2048.
2. Clock constraints used to achieve the performance numbers is: ACLK = 150 MHz.

The following table lists the CoreAXI4ProtoConv MM2S utilization when MM2S data width is 32 bits, MM2S command/status FIFO enable is 0, and MM2S data FIFO enable is 0.

Table 10-31. CoreAXI4ProtoConv's MM2S Utilization (MM2S Data Width (in bits) = 32, MM2S Command/Status FIFO Enable = 0 and MM2S Data FIFO Enable = 0)

Device Details		Resources			Performance	RAMs	
Family	Device	4LUT	DFF	Logic Elements	ACLK	LSRAM	uSRAM
PolarFire® SoC	MPFS250T	746	368	753	340	0	0
PolarFire	MPF300T	746	368	753	310	0	0
RT PolarFire	RTPF500T	746	368	753	310	0	0
RTG4™	RT4G150	771	368	785	180	0	0

.....continued

Device Details		Resources			Performance	RAMs	
Family	Device	4LUT	DFF	Logic Elements	ACLK	LSRAM	uSRAM
SmartFusion® 2	M2S150TS	1312	807	1361	175	0	0
IGLOO® 2	M2GL150TS	1312	807	1361	175	0	0



Important: The data in the preceding table is achieved using default synthesis and layout settings.

- The following top-level configuration GUI parameters have been modified from their default values:
 - MM2S Data FIFO Enable = 0
 - MM2S Command/Status FIFO Enable = 0
 - MM2S Data Width (in bits) = 32
- Clock constraints used to achieve the performance numbers is: ACLK = 150 MHz

The following table lists the CoreAXI4ProtoConv MM2S utilization when MM2S data width is 64 bits, MM2S command/status FIFO enable is 0, and MM2S data FIFO enable is 0.

Table 10-32. CoreAXI4ProtoConv's MM2S Utilization (MM2S Data Width (in bits) = 64, MM2S Command/Status FIFO Enable = 0 and MM2S Data FIFO Enable = 0)

Device Details		Resources			Performance	RAMs	
Family	Device	4LUT	DFF	Logic Elements	ACLK	LSRAM	uSRAM
PolarFire® SoC	MPFS250T	707	367	723	340	0	0
PolarFire	MPF300T	707	367	723	320	0	0
RT PolarFire	RTPF500T	707	367	723	320	0	0
RTG4™	RT4G150	699	367	719	175	0	0
SmartFusion® 2	M2S150TS	1248	811	1297	180	0	0
IGLOO® 2	M2GL150TS	1248	811	1297	180	0	0



Important: The data in the preceding table is achieved using default synthesis and layout settings.

- The following top-level configuration GUI parameters have been modified from their default values:
 - MM2S Data FIFO Enable = 0
 - MM2S Command/Status FIFO Enable = 0
 - MM2S Data Width (in bits) = 64
- Clock constraints used to achieve the performance numbers is: ACLK = 150 MHz

The following table lists the CoreAXI4ProtoConv MM2S utilization when MM2S data width is 128 bits, MM2S command/status FIFO enable is 0, and MM2S data FIFO enable is 0.

Table 10-33. CoreAXI4ProtoConv's MM2S Utilization (MM2S Data Width (in bits) = 128, MM2S Command/Status FIFO Enable = 0 and MM2S Data FIFO Enable = 0)

Device Details		Resources			Performance	RAMs	
Family	Device	4LUT	DFF	Logic Elements	ACLK	LSRAM	uSRAM
PolarFire [®] SoC	MPFS250T	701	366	719	310	0	0
PolarFire	MPF300T	701	366	719	310	0	0
RT PolarFire	RTPF500T	701	366	719	310	0	0
RTG4 [™]	RT4G150	739	366	757	195	0	0
SmartFusion [®] 2	M2S150TS	1234	736	1266	175	0	0
IGLOO [®] 2	M2GL150TS	1234	736	1266	175	0	0



Important: The data in the preceding table is achieved using default synthesis and layout settings.

- The following top-level configuration GUI parameters have been modified from their default values:
 - MM2S Data FIFO Enable = 0
 - MM2S Command/Status FIFO Enable = 0
 - MM2S Data Width (in bits) = 128
- Clock constraints used to achieve the performance numbers is: ACLK = 150 MHz

The following table lists the CoreAXI4ProtoConv MM2S utilization when MM2S data width is 256 bits, MM2S command/status FIFO enable is 0, and MM2S data FIFO enable is 0.

Table 10-34. CoreAXI4ProtoConv's MM2S Utilization (MM2S Data Width (in bits) = 256, MM2S Command/Status FIFO Enable = 0 and MM2S Data FIFO Enable = 0)

Device Details		Resources			Performance	RAMs	
Family	Device	4LUT	DFF	Logic Elements	ACLK	LSRAM	uSRAM
PolarFire [®] SoC	MPFS250T	704	365	714	310	0	0
PolarFire	MPF300T	704	365	714	310	0	0
RT PolarFire	RTPF500T	704	365	714	310	0	0
RTG4 [™]	RT4G150	743	365	753	195	0	0
SmartFusion [®] 2	M2S150TS	1221	746	1275	165	0	0
IGLOO [®] 2	M2GL150TS	1221	746	1275	165	0	0

 **Important:** The data in the preceding table is achieved using default synthesis and layout settings.

- The following top-level configuration GUI parameters have been modified from their default values:
 - MM2S Data FIFO Enable = 0
 - MM2S Data Width (in bits) = 256
 - MM2S Command/Status FIFO Enable = 0
- Clock constraints used to achieve the performance numbers is: ACLK = 150 MHz

The following table lists the CoreAXI4ProtoConv MM2S utilization when MM2S data width is 512 bits, MM2S command/status FIFO enable is 0, and MM2S data FIFO enable is 0.

Table 10-35. CoreAXI4ProtoConv's MM2S Utilization (MM2S Data Width (in bits) = 512, MM2S Command/Status FIFO Enable = 0 and MM2S Data FIFO Enable = 0)

Device Details		Resources			Performance	RAMs	
Family	Device	4LUT	DFF	Logic Elements	ACLK	LSRAM	uSRAM
PolarFire [®] SoC	MPFS250T	759	364	783	330	0	0
PolarFire	MPF300T	759	364	783	330	0	0
RT PolarFire	RTPF500T	759	364	783	330	0	0
RTG4 [™]	RT4G150	756	364	764	190	0	0
SmartFusion [®] 2	M2S150TS	1227	740	1261	170	0	0
IGLOO [®] 2	M2GL150TS	1227	740	1261	170	0	0

 **Important:** The data in the preceding table is achieved using default synthesis and layout settings.

- The following top-level configuration GUI parameters have been modified from their default values:
 - MM2S Data FIFO Enable = 0
 - MM2S Command/Status FIFO Enable = 0
 - MM2S Data Width (in bits) = 512
- Clock constraints used to achieve the performance numbers is: ACLK = 150 MHz

11. Glossary [\(Ask a Question\)](#)

Following are the list of terms and definitions used in the document.

Table 11-1. Terms and Definitions

Term	Definition
AXI4	Advanced eXtensible Interface v4
AXI4S	Advanced eXtensible Interface v4 Stream
DFF	D Flip-Flop
ECC	Error Correction Code
FIFO	First in First Out
GUI	Graphical User Interface
LSRAM	Large Synchronous Random Accessible Memory
LUT	Look Up Table
μSRAM	Micro Synchronous Random Accessible Memory
MM2S	Memory Mapped to Stream
S2MM	Stream to Memory Mapped

12. Resolved Issues [\(Ask a Question\)](#)

The following table lists all the resolved issues for the various CoreAXI4ProtoConv releases.

Table 12-1. Resolved Issues

Release	Description
3.0	• Addressed command ID alignment in both S2MM and MM2S registers. • Updated block diagrams to include specific interface names and prefixes, and created separate diagrams for different FIFO configurations. • Consolidated the two AXI4-Lite interfaces into a single AXI4-Lite interface.
2.0	Initial Release

13. Revision History [\(Ask a Question\)](#)

The revision history describes the changes that were implemented in the document. The changes are listed by revision, starting with the current publication.

Table 13-1. Revision History

Revision	Date	Description
D	01/2025	The following is the list of changes in revision D of the document: - Updated Change Log Information for v3.0 in the Introduction section. - Updated description for the S2MM_LEN and MM2S_LEN registers.
C	12/2024	The following is the list of changes in revision C of the document: Updated S2MM_LEN and MM2S_LEN registers.
B	11/2024	The following is the list of changes in revision B of the document: - Updated changes related to CoreAXI4ProtoConv v3.0. - Updated all the block diagrams related to S2MM and M2SS macro. - Added Programming Sequence section. - Updated SmartDesign figures in the SmartDesign section. - Updated GUI configuration figures in the Configuring the CoreAXI4ProtoConv section. - Updated information related to Simulation in the Simulation Flow section. - Added Resolved Issues section.
A	10/2022	Initial Revision.

Microchip FPGA Support

Microchip FPGA products group backs its products with various support services, including Customer Service, Customer Technical Support Center, a website, and worldwide sales offices. Customers are suggested to visit Microchip online resources prior to contacting support as it is very likely that their queries have been already answered.

Contact Technical Support Center through the website at www.microchip.com/support. Mention the FPGA Device Part number, select appropriate case category, and upload design files while creating a technical support case.

Contact Customer Service for non-technical product support, such as product pricing, product upgrades, update information, order status, and authorization.

- From North America, call **800.262.1060**
- From the rest of the world, call **650.318.4460**
- Fax, from anywhere in the world, **650.318.8044**

Microchip Information

Trademarks

The “Microchip” name and logo, the “M” logo, and other names, logos, and brands are registered and unregistered trademarks of Microchip Technology Incorporated or its affiliates and/or subsidiaries in the United States and/or other countries (“Microchip Trademarks”). Information regarding Microchip Trademarks can be found at <https://www.microchip.com/en-us/about/legal-information/microchip-trademarks>.

ISBN: 979-8-3371-0382-2

Legal Notice

This publication and the information herein may be used only with Microchip products, including to design, test, and integrate Microchip products with your application. Use of this information in any other manner violates these terms. Information regarding device applications is provided only for your convenience and may be superseded by updates. It is your responsibility to ensure that your application meets with your specifications. Contact your local Microchip sales office for additional support or, obtain additional support at www.microchip.com/en-us/support/design-help/client-support-services.

THIS INFORMATION IS PROVIDED BY MICROCHIP “AS IS”. MICROCHIP MAKES NO REPRESENTATIONS OR WARRANTIES OF ANY KIND WHETHER EXPRESS OR IMPLIED, WRITTEN OR ORAL, STATUTORY OR OTHERWISE, RELATED TO THE INFORMATION INCLUDING BUT NOT LIMITED TO ANY IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS FOR A PARTICULAR PURPOSE, OR WARRANTIES RELATED TO ITS CONDITION, QUALITY, OR PERFORMANCE.

IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE, INCIDENTAL, OR CONSEQUENTIAL LOSS, DAMAGE, COST, OR EXPENSE OF ANY KIND WHATSOEVER RELATED TO THE INFORMATION OR ITS USE, HOWEVER CAUSED, EVEN IF MICROCHIP HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP’S TOTAL LIABILITY ON ALL CLAIMS IN ANY WAY RELATED TO THE INFORMATION OR ITS USE WILL NOT EXCEED THE AMOUNT OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THE INFORMATION.

Use of Microchip devices in life support and/or safety applications is entirely at the buyer’s risk, and the buyer agrees to defend, indemnify and hold harmless Microchip from any and all damages, claims, suits, or expenses resulting from such use. No licenses are conveyed, implicitly or otherwise, under any Microchip intellectual property rights unless otherwise stated.

Microchip Devices Code Protection Feature

Note the following details of the code protection feature on Microchip products:

- Microchip products meet the specifications contained in their particular Microchip Data Sheet.
- Microchip believes that its family of products is secure when used in the intended manner, within operating specifications, and under normal conditions.
- Microchip values and aggressively protects its intellectual property rights. Attempts to breach the code protection features of Microchip products are strictly prohibited and may violate the Digital Millennium Copyright Act.
- Neither Microchip nor any other semiconductor manufacturer can guarantee the security of its code. Code protection does not mean that we are guaranteeing the product is “unbreakable”. Code protection is constantly evolving. Microchip is committed to continuously improving the code protection features of our products.